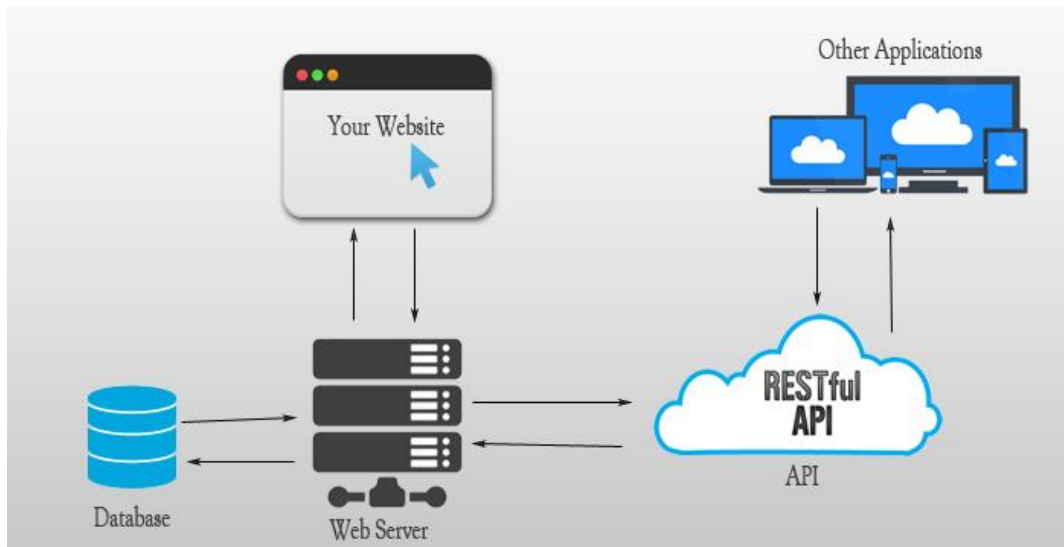


API-WEB-MYSQL



En este proyecto trabajamos en la construcción de una aplicación completa que combina una API REST, un sistema web basado en plantillas Jinja2 y una base de datos MySQL. Integramos backend, frontend y persistencia de datos en un mismo entorno, demostrando que somos capaces de gestionar información desde distintos puntos de acceso. Nuestro objetivo es mostrar cómo una API puede convivir con páginas web dinámicas, compartiendo la misma lógica y los mismos datos, tal como lo hacen las aplicaciones modernas que utilizan múltiples capas tecnológicas.

Vamos a desarrollar una aplicación que permita gestionar registros desde dos frentes complementarios: por un lado, una API REST que expone los datos en formato JSON para que puedan ser consumidos por cualquier cliente externo; y por otro lado, un conjunto de páginas web generadas con Jinja2 que nos permiten interactuar con esos mismos datos de forma visual mediante formularios y vistas HTML. Esto significa que podremos crear, listar, editar y eliminar registros tanto desde la API como desde la interfaz web, demostrando que ambas capas trabajan sobre la misma lógica y la misma información. Además, añadiremos estilos, fragmentos reutilizables y una estructura organizada de plantillas para mantener un diseño coherente en todas las páginas. Finalmente, conectaremos todo con una base de datos MySQL para asegurar que los datos se almacenen de manera persistente y profesional. Con este enfoque demostramos que dominamos la integración completa entre API, web y base de datos dentro de un mismo proyecto funcional.

Podemos imaginar este proyecto como un aeropuerto: la API sería la pista donde despegan y aterrizan los datos en formato técnico, la base de datos sería el hangar donde almacenamos toda la información, y las páginas web serían la terminal donde los usuarios ven, consultan y gestionan todo de forma visual y cómoda. Cada parte cumple un rol distinto, pero todas trabajan juntas para que el sistema funcione de manera fluida y coordinada.

1. AÑADIR PÁGINAS WEB CON JINJA2 A LA API REST DE LA PRÁCTICA ANTERIOR.

main.py

En main.py centralizamos toda la lógica de nuestra aplicación, ya que aquí definimos la API REST, configuramos las rutas que devuelven páginas HTML mediante Jinja2 y gestionamos el CRUD completo sobre los datos. Desde este archivo controlamos cómo se reciben las peticiones, cómo se procesan y qué respuestas se envían tanto en formato JSON como en vistas web. Además, organizamos la estructura general del proyecto, conectamos las plantillas, servimos los archivos estáticos y coordinamos el flujo entre la interfaz web y la lógica interna. En resumen, main.py actúa como el núcleo que integra la API, la web y los datos en un solo punto.

```
main.py X
src > main.py > ...
1  from fastapi import FastAPI, Request, HTTPException, Form
2  from fastapi.responses import HTMLResponse, RedirectResponse
3  from fastapi.templating import Jinja2Templates
4  from fastapi.staticfiles import StaticFiles
5  from pydantic import BaseModel
6
7  app = FastAPI()
8
9  templates = Jinja2Templates(directory="templates")
10 app.mount("/static", StaticFiles(directory="static"), name="static")
11
12
13 class Coche(BaseModel):
14     id: int
15     modelo: str
16     anio: int
17     precio: float
18
19
20 coches = [
21     Coche(id=1, modelo="Audi A1", anio=2021, precio=22000),
22     Coche(id=2, modelo="Audi A3", anio=2022, precio=28000),
23     Coche(id=3, modelo="Audi A4", anio=2023, precio=39000),
24     Coche(id=4, modelo="Audi A5", anio=2023, precio=45000),
25     Coche(id=5, modelo="Audi A6", anio=2021, precio=52000),
26     Coche(id=6, modelo="Audi A7", anio=2022, precio=68000),
27     Coche(id=7, modelo="Audi A8", anio=2023, precio=89000),
28     Coche(id=8, modelo="Audi Q3", anio=2022, precio=35000),
29     Coche(id=9, modelo="Audi Q5", anio=2024, precio=61000),
30     Coche(id=10, modelo="Audi e-tron", anio=2023, precio=72000),
31 ]
32
33
34
35 @app.get("/", response_class=HTMLResponse)
```

index.html

En index.html construimos la página principal de la aplicación, donde presentamos la interfaz inicial que el usuario ve al entrar al sistema. Aquí mostramos el banner, el mensaje de bienvenida y la galería de modelos destacados, utilizando fragmentos reutilizables para mantener un diseño limpio y coherente. Esta página sirve como punto de entrada a las demás secciones del proyecto, guiando al usuario hacia el listado de registros y las funcionalidades del CRUD. Con esta plantilla demostramos cómo Jinja2 nos permite generar contenido dinámico y estructurado dentro de una web integrada con FastAPI.

```
<> index.html X
src > templates > <> index.html > h1
1  {% extends "fragments/base.html" %}
2
3  {% block title %}Inicio{% endblock %}
4
5  {% block content %}
6
7  <h1>Concesionario Oficial Audi</h1>
8  <p class="intro-text">
9      Descubramos juntos nuestra selección de vehículos Audi: diseño, tecnología y rendimiento.
10     <br>
11     <br>
12 </p>
13
14 <div class="hero-banner">
```

base.html

En base.html definimos la estructura general que comparten todas las páginas del proyecto, incluyendo la cabecera, el menú de navegación, el contenedor principal y el pie de página. Gracias a esta plantilla base evitamos repetir código y garantizamos que todas las vistas mantengan el mismo estilo y diseño. Utilizamos bloques de Jinja2 para permitir que cada página inserte su contenido específico sin perder la coherencia visual. Este archivo es esencial para mantener una arquitectura ordenada y profesional en todo el sistema.

```
<> base.html X
src > templates > fragments > <> base.html > html
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>{% block title %}{% endblock %}</title>
7
8      <link href="https://fonts.googleapis.com/css2?family=Inter:wght@300;400;600&display=block" rel="stylesheet">
9      <link rel="stylesheet" href="/static/css/styles.css">
10 </head>
11
12 <body>
13
```

coches.html

En coches.html mostramos el listado completo de registros almacenados, permitiendo visualizar de forma clara todos los elementos gestionados por la aplicación. Aquí utilizamos una tabla generada dinámicamente con Jinja2 para recorrer los datos y mostrar cada registro junto con sus acciones correspondientes: ver, editar y eliminar. Esta página representa la parte central del CRUD y demuestra cómo combinamos datos internos con plantillas HTML para generar contenido dinámico.

```
coches.html X
src > templates > coches.html > table > tr
1  {% extends "fragments/base.html" %}
2
3  {% block title %}Listado de coches{% endblock %}
4
5  {% block content %}
6  <h1>Listado de coches Audi</h1>
7
8  <table>
9      <tr>
10         <th>ID</th>
11         <th>Modelo</th>
12         <th>Año</th>
13         <th>Precio</th>
14         <th>Acciones</th>
15     </tr>
16
17     {% for coche in coches %}
18     <tr>
19         <td>{{ coche.id }}</td>
20         <td>{{ coche.modelo }}</td>
21         <td>{{ coche.anio }}</td>
22         <td>{{ coche.precio }} €</td>
23         <td>
24             <a class="btn btn-small" href="/coches/{{ coche.id }}">Ver</a>
25             <a class="btn btn-small" href="/coches/editar/{{ coche.id }}">Editar</a>
26             <a class="btn btn-small btn-delete" href="/coches/eliminar/{{ coche.id }}">
27         </td>
28     </tr>
29     {% endfor %}
30 </table>
31
32 <br>
33 <br>
```

crear_coche.html

En crear_coche.html implementamos el formulario que permite añadir nuevos registros al sistema. Esta plantilla recibe datos desde el servidor, muestra mensajes de confirmación cuando se crea un elemento y envía la información mediante un formulario POST. Con esta página demostramos cómo Jinja2 interactúa con FastAPI para procesar formularios, validar entradas y actualizar la información disponible en la aplicación.

editar_coche.html

En `editar_coche.html` construimos la interfaz que permite modificar un registro existente. La plantilla recibe los datos actuales del elemento seleccionado y los muestra en un formulario editable, permitiendo actualizar la información de manera sencilla. Además, gestionamos mensajes de éxito para confirmar que los cambios se han guardado correctamente. Esta página evidencia cómo combinamos rutas dinámicas, formularios y plantillas para completar el ciclo del CRUD.

eliminar_coche.html

En `eliminar_coche.html` presentamos una vista de confirmación antes de borrar un registro, evitando eliminaciones accidentales. La plantilla muestra los datos del elemento seleccionado y ofrece opciones claras para confirmar o cancelar la acción. Este archivo demuestra cómo gestionamos operaciones sensibles dentro de la aplicación, manteniendo una interacción segura y controlada con el usuario.

coche_detalle.html

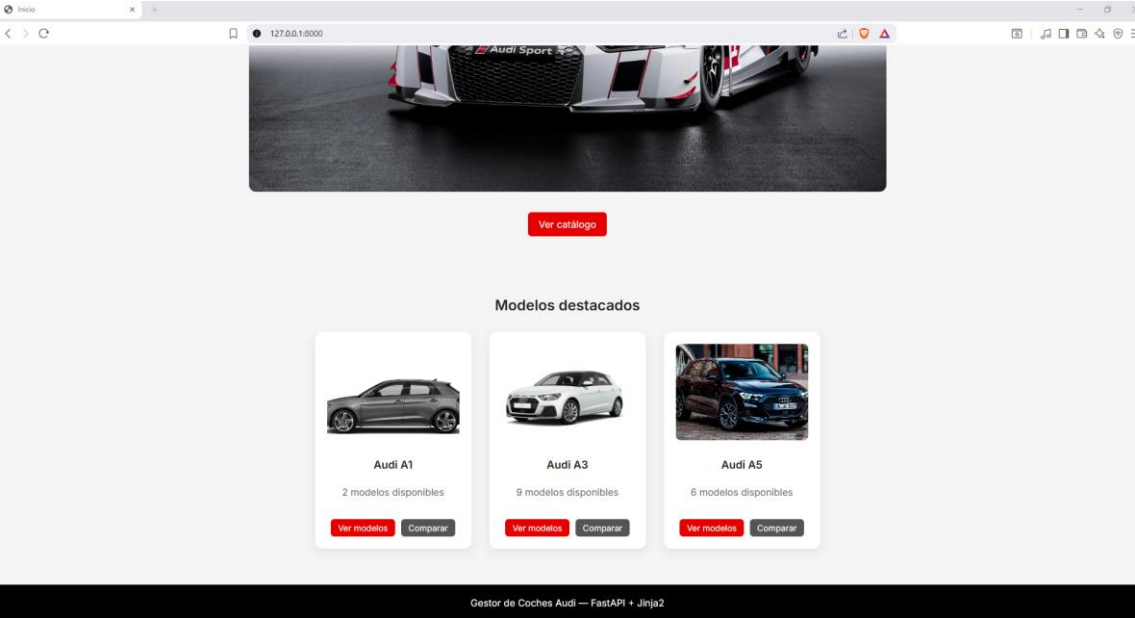
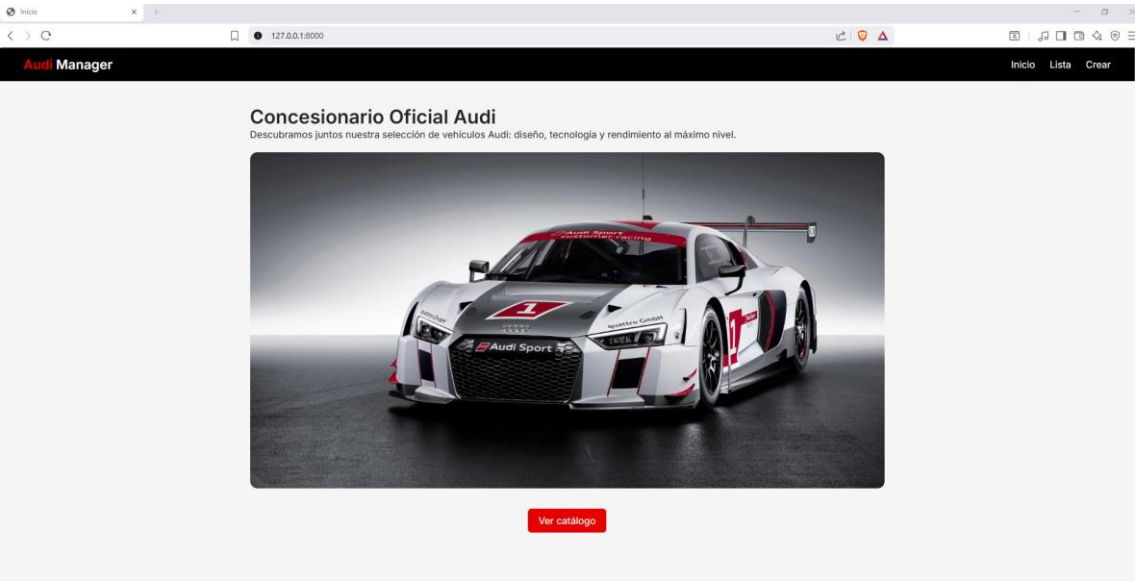
En `coche_detalle.html` mostramos la información detallada de un registro específico, permitiendo al usuario consultar sus datos de manera individual. Esta plantilla recibe el objeto seleccionado desde el servidor y lo presenta de forma organizada, junto con accesos directos para editarlo o eliminarlo. Con esta vista completamos la parte visual del CRUD, ofreciendo una experiencia más completa y profesional.

styles.css

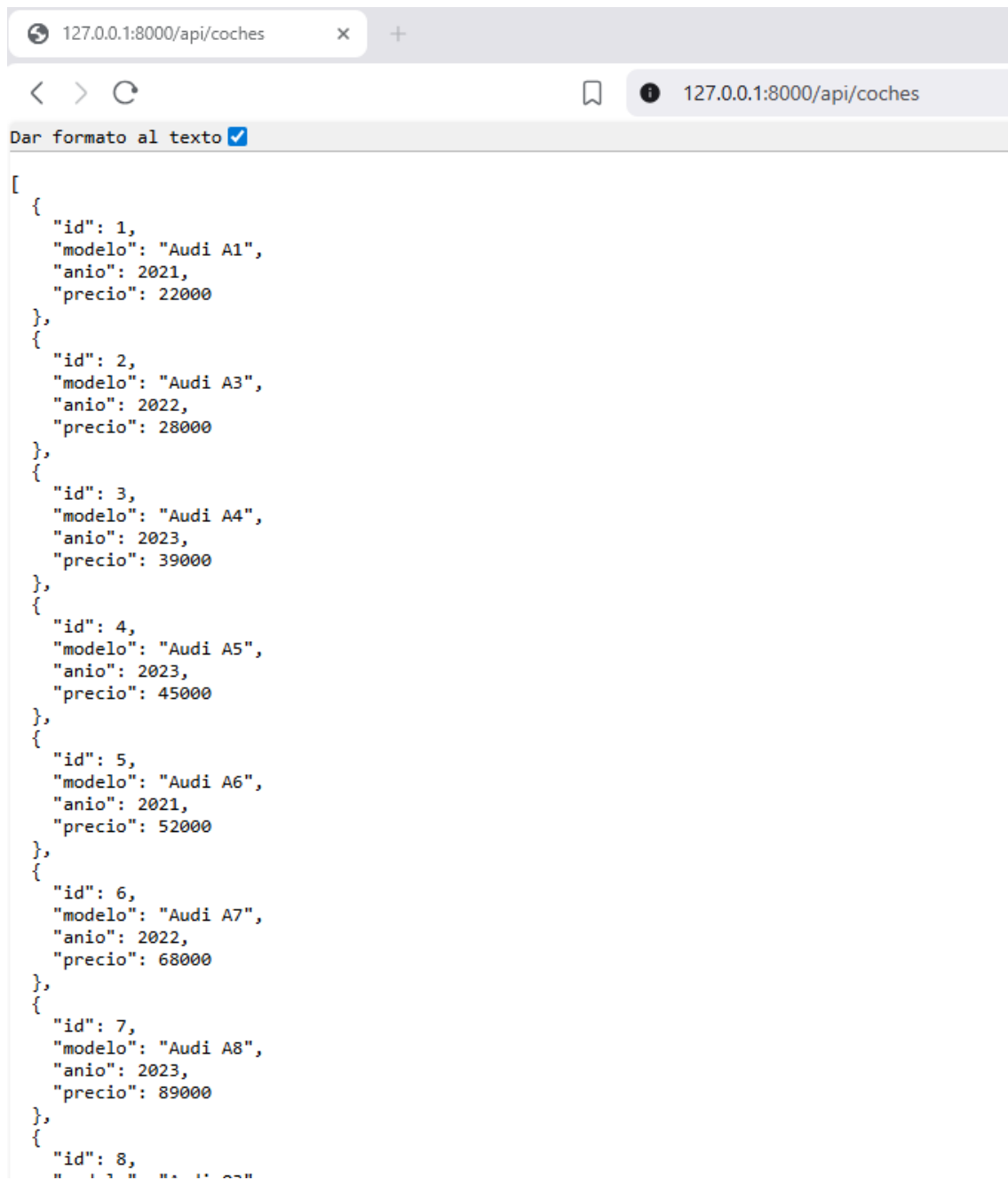
En `styles.css` definimos toda la apariencia visual de la aplicación, incluyendo colores, tipografías, botones, tablas, formularios y la galería de modelos. Este archivo nos permite mantener un diseño coherente en todas las páginas y separar claramente la lógica del estilo. Gracias a esta hoja de estilos logramos que la aplicación tenga una presentación moderna, limpia y alineada con la estética de la marca Audi.

Links:

- **<http://127.0.0.1:8000/>** Es la página principal generada con Jinja2, donde mostramos la interfaz inicial del proyecto.
- **<http://127.0.0.1:8000/coches>** Es la página que muestra el listado completo de registros usando plantillas Jinja2.
- **<http://127.0.0.1:8000/api/coches>** Es el endpoint de la API REST que devuelve los datos en formato JSON.



ID	Modelo	Año	Precio	Acciones
1	Audi A1	2021	22000.0 €	Ver Editar Eliminar
2	Audi A3	2022	28000.0 €	Ver Editar Eliminar
3	Audi A4	2023	39000.0 €	Ver Editar Eliminar
4	Audi A5	2023	45000.0 €	Ver Editar Eliminar
5	Audi A6	2021	52000.0 €	Ver Editar Eliminar
6	Audi A7	2022	68000.0 €	Ver Editar Eliminar
7	Audi A8	2023	89000.0 €	Ver Editar Eliminar
8	Audi Q3	2022	35000.0 €	Ver Editar Eliminar
9	Audi Q5	2024	61000.0 €	Ver Editar Eliminar
10	Audi e-tron	2023	72000.0 €	Ver Editar Eliminar



The screenshot shows a web browser window with the address bar displaying '127.0.0.1:8000/api/coches'. The page content shows a JSON array of car data. A checkbox labeled 'Dar formato al texto' is checked. The JSON data is as follows:

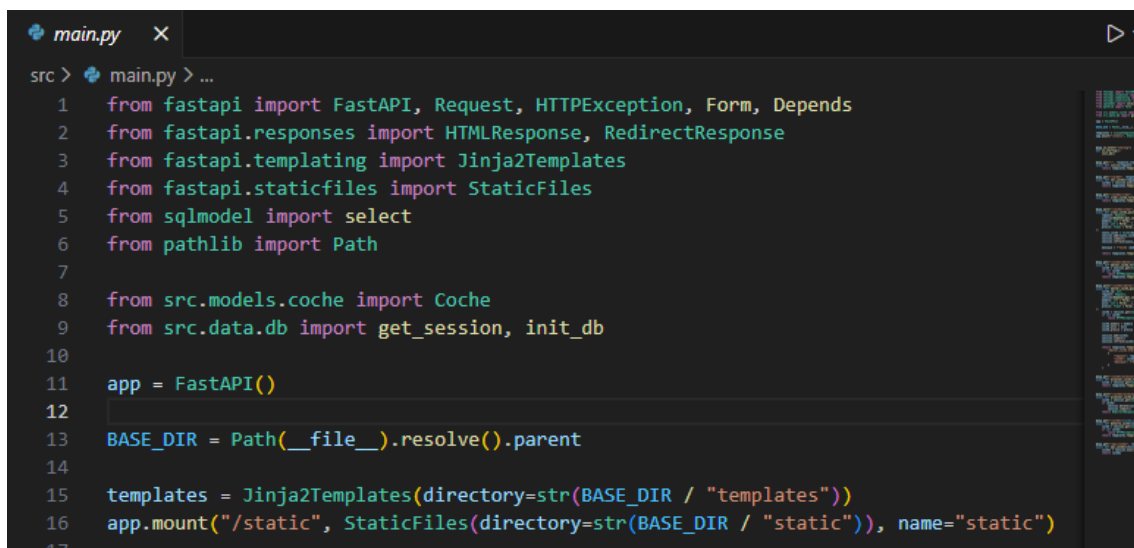
```
[
  {
    "id": 1,
    "modelo": "Audi A1",
    "anio": 2021,
    "precio": 22000
  },
  {
    "id": 2,
    "modelo": "Audi A3",
    "anio": 2022,
    "precio": 28000
  },
  {
    "id": 3,
    "modelo": "Audi A4",
    "anio": 2023,
    "precio": 39000
  },
  {
    "id": 4,
    "modelo": "Audi A5",
    "anio": 2023,
    "precio": 45000
  },
  {
    "id": 5,
    "modelo": "Audi A6",
    "anio": 2021,
    "precio": 52000
  },
  {
    "id": 6,
    "modelo": "Audi A7",
    "anio": 2022,
    "precio": 68000
  },
  {
    "id": 7,
    "modelo": "Audi A8",
    "anio": 2023,
    "precio": 89000
  },
  {
    "id": 8,
    "modelo": "Audi A9",
    "anio": 2024,
    "precio": 100000
  }
]
```

En esta parte del proyecto cumplimos con el primer punto porque demostramos claramente la integración de páginas web generadas con Jinja2 dentro de una API desarrollada con FastAPI, permitiendo que ambas capas convivan y trabajen sobre la misma lógica. Implementamos rutas que devuelven vistas HTML, utilizamos plantillas reutilizables, incorporamos formularios funcionales y mostramos datos dinámicos en la interfaz web, todo ello sin depender aún de bases de datos externas ni configuraciones adicionales. Con este enfoque dejamos evidencia de que somos capaces de combinar una API REST con un sistema web completo dentro del mismo proyecto, cumpliendo exactamente con el requisito solicitado y mostrando un dominio sólido de la estructura y funcionamiento de aplicaciones híbridas API-Web.

2. HACER QUE FUNCIONE CON UNA BASE DE DATOS MYSQL EN LOCALHOST.

main.py

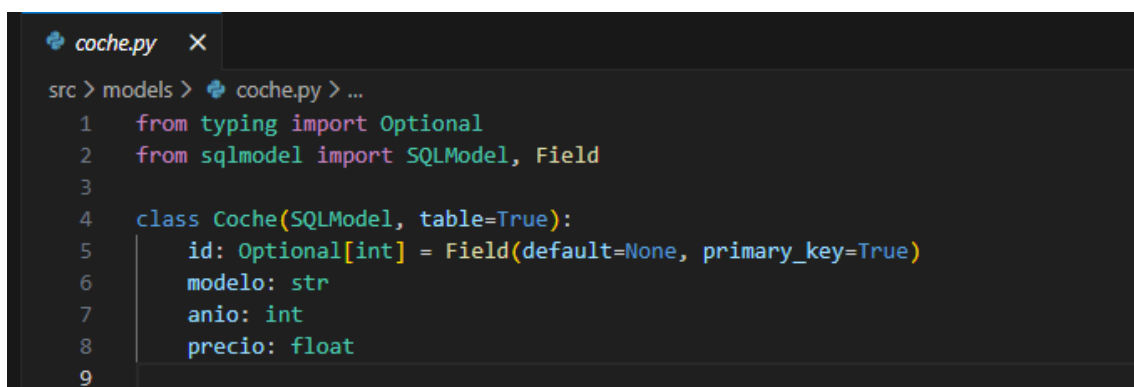
En main.py concentramos toda la lógica principal de nuestra aplicación, y por eso es el archivo que más hemos modificado en esta fase. Aquí reemplazamos la lista en memoria por consultas reales a MySQL, añadimos la dependencia de sesión para que cada ruta pueda comunicarse con la base de datos y adaptamos todas las operaciones del CRUD para que trabajen directamente con datos persistentes. También configuramos la carga de plantillas, los archivos estáticos y el evento de inicio que inicializa la base de datos. En resumen, modificamos main.py para convertir una aplicación local y temporal en un sistema completo conectado a MySQL.



```
main.py X
src > main.py > ...
1 from fastapi import FastAPI, Request, HTTPException, Form, Depends
2 from fastapi.responses import HTMLResponse, RedirectResponse
3 from fastapi.templating import Jinja2Templates
4 from fastapi.staticfiles import StaticFiles
5 from sqlmodel import select
6 from pathlib import Path
7
8 from src.models.coche import Coche
9 from src.data.db import get_session, init_db
10
11 app = FastAPI()
12
13 BASE_DIR = Path(__file__).resolve().parent
14
15 templates = Jinja2Templates(directory=str(BASE_DIR / "templates"))
16 app.mount("/static", StaticFiles(directory=str(BASE_DIR / "static")), name="static")
17
```

models/coche.py

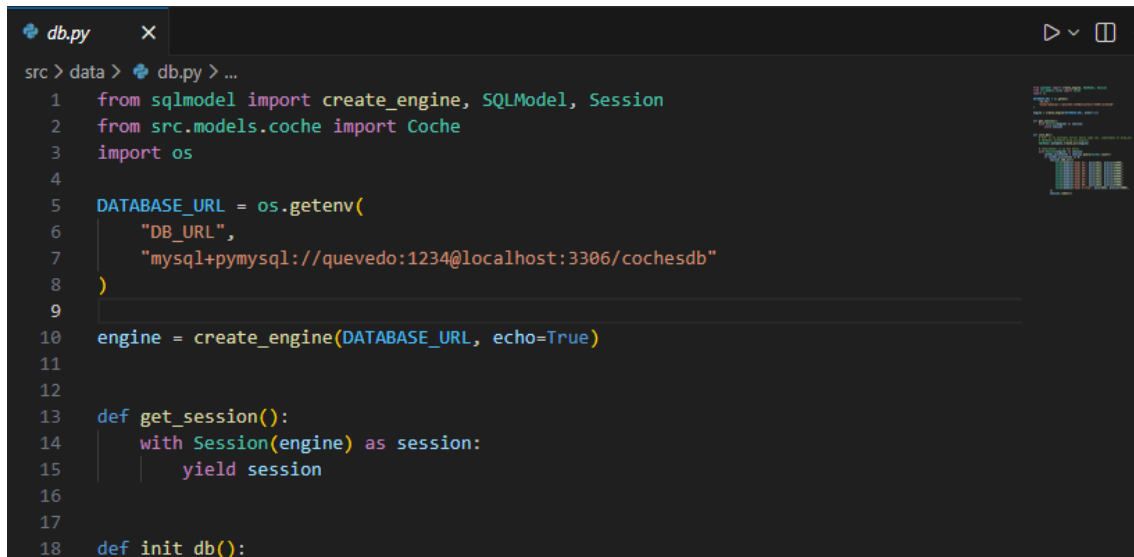
En coche.py definimos el modelo de datos utilizando SQLAlchemy, especificando los campos, tipos y la estructura de la tabla que se creará en MySQL. Creamos este archivo para separar la definición del modelo del resto del código, siguiendo buenas prácticas de organización. Gracias a este modelo, FastAPI y SQLAlchemy pueden generar la tabla automáticamente, validar los datos y permitir que el CRUD funcione de forma tipada y coherente.



```
coche.py X
src > models > coche.py > ...
1 from typing import Optional
2 from sqlmodel import SQLModel, Field
3
4 class Coche(SQLModel, table=True):
5     id: Optional[int] = Field(default=None, primary_key=True)
6     modelo: str
7     anio: int
8     precio: float
9
```

data/db.py

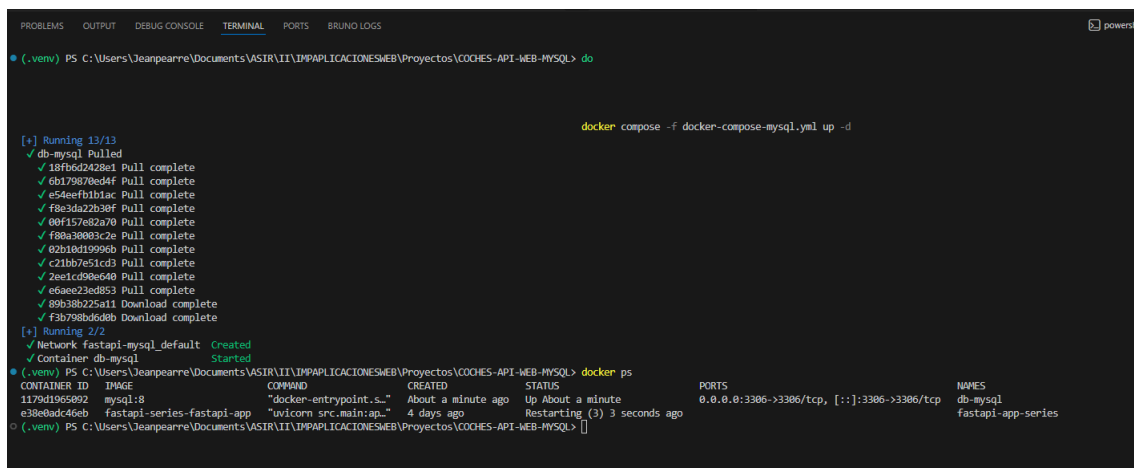
En db.py configuramos la conexión con MySQL y creamos el motor que permite a la aplicación comunicarse con la base de datos. Aquí definimos la función `get_session()` para gestionar sesiones de `SQLModel` y añadimos `init_db()` para crear las tablas y cargar datos iniciales si la base de datos está vacía. Creamos este archivo para centralizar toda la lógica de conexión y evitar mezclarla con las rutas o las plantillas, manteniendo una estructura limpia y profesional.

A screenshot of a code editor window titled 'db.py'. The code is written in Python and defines a database connection and session management. It imports 'create_engine', 'SQLModel', and 'Session' from 'sqlmodel', and 'Coche' from 'src.models.coche'. It also imports 'os'. A constant 'DATABASE_URL' is set to 'mysql+pymysql://quevedo:1234@localhost:3306/cochesdb'. An 'engine' is created with 'create_engine(DATABASE_URL, echo=True)'. A 'get_session()' function is defined, which uses 'Session(engine)' as a context manager to yield a session. An 'init_db()' function is also defined.

```
src > data > db.py > ...
1  from sqlmodel import create_engine, SQLModel, Session
2  from src.models.coche import Coche
3  import os
4
5  DATABASE_URL = os.getenv(
6      "DB_URL",
7      "mysql+pymysql://quevedo:1234@localhost:3306/cochesdb"
8  )
9
10 engine = create_engine(DATABASE_URL, echo=True)
11
12
13 def get_session():
14     with Session(engine) as session:
15         yield session
16
17
18 def init_db():
```

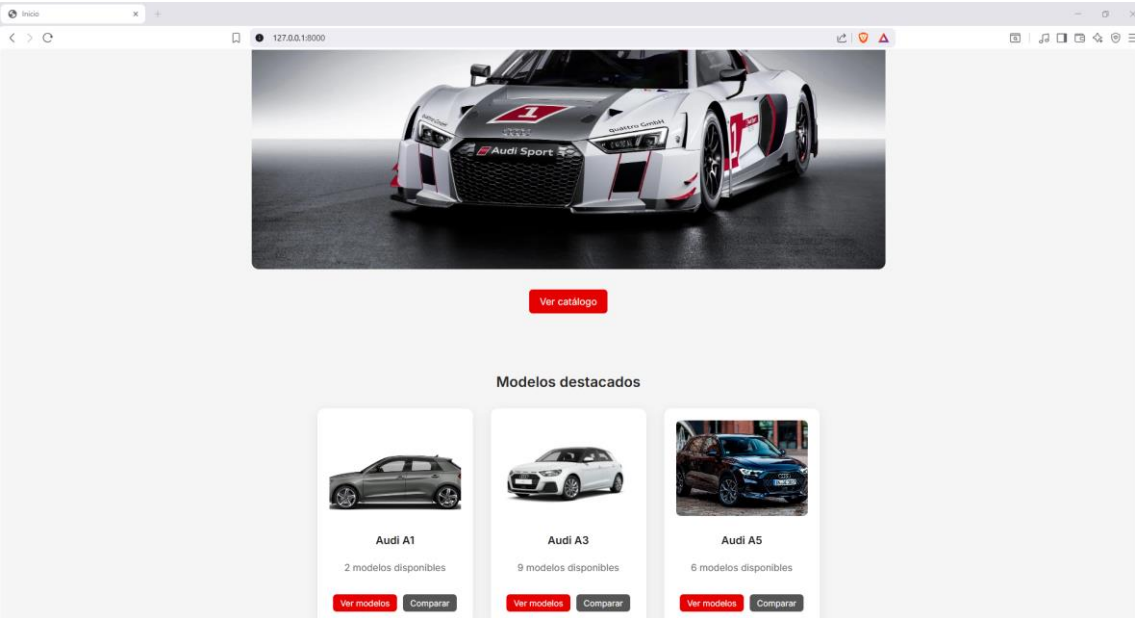
Docker compose (opción 1)

Usamos Docker en este proyecto porque nos permite levantar la base de datos MySQL de manera rápida, consistente y aislada, sin necesidad de instalarla directamente en nuestro sistema operativo. De esta forma, podemos asegurarnos de que todos trabajamos con la misma versión de MySQL y la misma configuración, evitando problemas de compatibilidad o diferencias entre equipos. Además, Docker nos facilita reiniciar, detener o eliminar la base de datos sin afectar nuestro entorno local, y nos prepara para, en pasos posteriores, ejecutar toda la aplicación en contenedores de manera ordenada y profesional.

A screenshot of a terminal window showing the output of a Docker Compose command. The command is 'docker compose -f docker-compose-mysql.yml up -d'. The output shows the progress of pulling and creating the 'db-mysql' service. It lists various layers being pulled and their status. Finally, it shows the 'db-mysql' container starting successfully. The terminal also shows the command 'docker ps' being run, which lists the running containers, including 'db-mysql' and 'fastapi-app-series'.

```
[+] Running 13/13
✓ db-mysql Pulled
✓ 18fb6d2428e1 Pull complete
✓ 6b179870ed4f Pull complete
✓ e54eefb1b14c Pull complete
✓ f8a3da22b30f Pull complete
✓ 00f157e82a70 Pull complete
✓ f8aa30083c2e Pull complete
✓ 02b10d1999eb Pull complete
✓ c21bb7e51cd3 Pull complete
✓ 2ee1cd90e649 Pull complete
✓ e69e023ed8c3 Pull complete
✓ 80b38b2725a1 Download complete
✓ f3b798bd6d8b Download complete
[+] Running 2/2
✓ Network fastapi-mysql_default Created
✓ Container db-mysql Started
(.venv) PS C:\Users\Jeanpearne\Documents\ASIR\I\INAPLICACIONES\WEB\Proyectos\COCHES-API-WEB-MYSQL> docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED          STATUS          PORTS                               NAMES
1179d1965892   mysql:8.0                                     "docker-entrypoint.s..." About a minute   Up About a minute   0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp   db-mysql
e38e8ad4c4eb   fastapi-series-fastapi-app                 "uvicorn src.main:app..." 4 days ago       Restarting (3) 3 seconds ago                                fastapi-app-series
(.venv) PS C:\Users\Jeanpearne\Documents\ASIR\I\INAPLICACIONES\WEB\Proyectos\COCHES-API-WEB-MYSQL>
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  BRUNO LOGS
INFO: Will watch for changes in these directories: ['C:\\Users\\Jeanpearre\\Documents\\ASIR\\II\\IMPPLICACIONESWEB\\Proyectos\\COCHES-API-WEB-MYSQL']
INFO: Will watch for changes in these directories: ['C:\\Users\\Jeanpearre\\Documents\\ASIR\\II\\IMPPLICACIONESWEB\\Proyectos\\COCHES-API-WEB-MYSQL']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [28092] using StatReload
INFO: Started server process [28932]
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [28092] using StatReload
INFO: Started server process [28932]
INFO: Waiting for application startup.
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [28092] using StatReload
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [28092] using StatReload
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [28092] using StatReload
INFO: Started server process [28932]
INFO: Waiting for application startup.
2026-02-01 01:55:19,934 INFO sqlalchemy.engine.Engine SELECT DATABASE()
2026-02-01 01:55:19,934 INFO sqlalchemy.engine.Engine [raw sql] {}
2026-02-01 01:55:19,939 INFO sqlalchemy.engine.Engine SELECT @@sql_mode
2026-02-01 01:55:19,941 INFO sqlalchemy.engine.Engine [raw sql] {}
2026-02-01 01:55:19,944 INFO sqlalchemy.engine.Engine SELECT @@lower_case_table_names
2026-02-01 01:55:19,945 INFO sqlalchemy.engine.Engine [raw sql] {}
2026-02-01 01:55:19,953 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2026-02-01 01:55:19,954 INFO sqlalchemy.engine.Engine DESCRIBE `cochesdb`.`coche`
2026-02-01 01:55:19,957 INFO sqlalchemy.engine.Engine [raw sql] {}
2026-02-01 01:55:19,993 INFO sqlalchemy.engine.Engine COMMIT
2026-02-01 01:55:20,001 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2026-02-01 01:55:20,014 INFO sqlalchemy.engine.Engine SELECT count(*) AS count_1
FROM (SELECT coche.id AS coche_id, coche.modelo AS coche_modelo, coche.anio AS coche_anio, coche.precio AS coche_precio
FROM coche) AS anon_1
2026-02-01 01:55:20,019 INFO sqlalchemy.engine.Engine [generated in 0.00431s] {}
2026-02-01 01:55:20,026 INFO sqlalchemy.engine.Engine ROLLBACK
INFO: Application startup complete.
[]
```



Lista de coches

127.0.0.1:8000/coches

Audi Manager Inicio Lista Crear

Listado de coches Audi

ID	Modelo	Año	Precio	Acciones
1	Audi A1	2021	22000.0 €	Ver Editar Eliminar
2	Audi A3	2022	28000.0 €	Ver Editar Eliminar
3	Audi A4	2023	39000.0 €	Ver Editar Eliminar
4	Audi A5	2023	45000.0 €	Ver Editar Eliminar
5	Audi A6	2021	52000.0 €	Ver Editar Eliminar
6	Audi A7	2022	68000.0 €	Ver Editar Eliminar
7	Audi A8	2023	89000.0 €	Ver Editar Eliminar
8	Audi Q3	2022	35000.0 €	Ver Editar Eliminar
9	Audi Q5	2024	61000.0 €	Ver Editar Eliminar
10	Audi e-tron	2023	72000.0 €	Ver Editar Eliminar

mysql portable (opción 2)

En este proyecto hemos utilizado MySQL Portable porque nos permite trabajar con una base de datos completa sin necesidad de instalar servicios permanentes en el sistema operativo. Esta versión portátil nos ofrece la ventaja de poder ejecutar MySQL de forma local, ligera y totalmente controlada, iniciando y deteniendo el servidor cuando lo necesitamos. Gracias a ello, podemos desarrollar, probar y gestionar la base de datos del proyecto sin modificar la configuración del equipo ni depender de servicios externos, manteniendo un entorno limpio, reproducible y fácil de transportar entre diferentes máquinas.

```
MySQL (cochesdb)>SHOW DATABASES;
+-----+
| Database |
+-----+
| cochesdb |
| information_schema |
| mysql |
| performance_schema |
| sys |
+-----+
5 rows in set (0.00 sec)

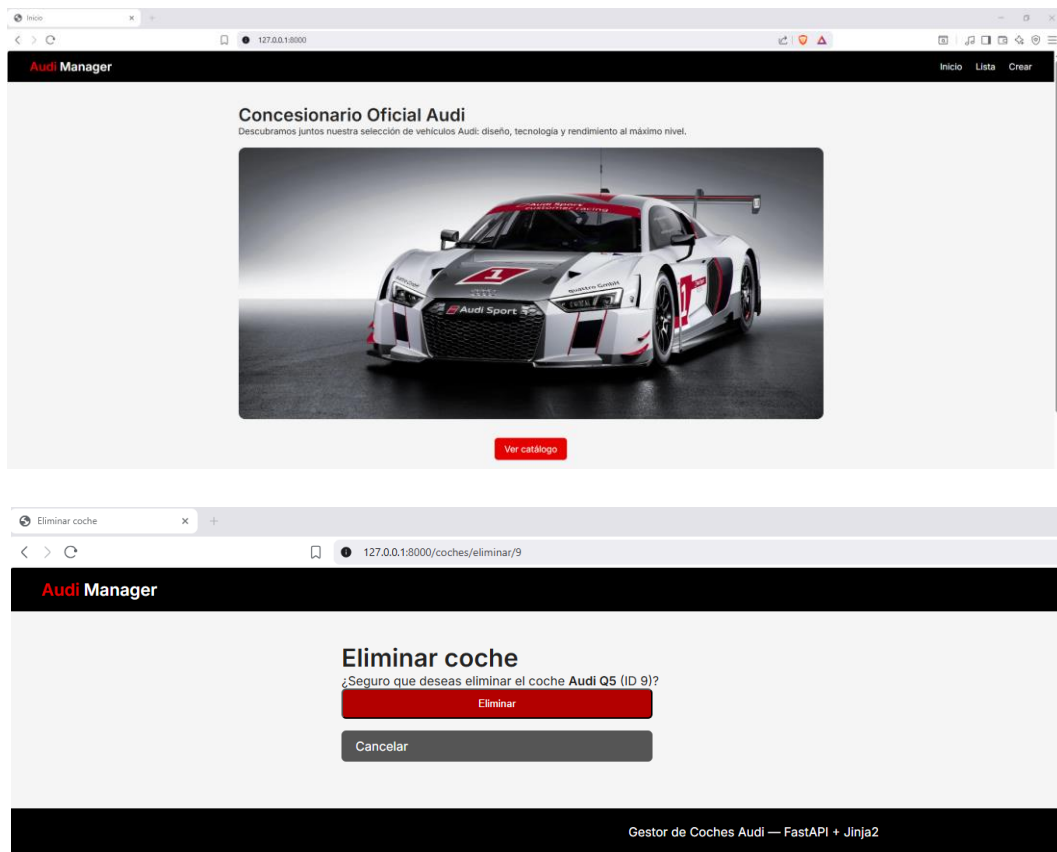
MySQL (cochesdb)>SELECT * FROM coche;
+----+-----+-----+-----+
| id | modelo | anio | precio |
+----+-----+-----+-----+
| 1 | Audi A1 | 2021 | 22000 |
| 2 | Audi A3 | 2022 | 28000 |
+----+-----+-----+-----+
```

```

| performance_schema |
| sys |
+-----+
5 rows in set (0.00 sec)

MySQL (cochesdb)>SELECT * FROM coche;
+----+-----+-----+-----+
| id | modelo | anio | precio |
+----+-----+-----+-----+
| 1 | Audi A1 | 2021 | 22000 |
| 2 | Audi A3 | 2022 | 28000 |
| 3 | Audi A4 | 2023 | 39000 |
| 4 | Audi A5 | 2023 | 45000 |
| 5 | Audi A6 | 2021 | 52000 |
| 6 | Audi A7 | 2022 | 68000 |
| 7 | Audi A8 | 2023 | 89000 |
| 8 | Audi Q3 | 2022 | 35000 |
| 9 | Audi Q5 | 2024 | 61000 |
+----+-----+-----+-----+
9 rows in set (0.00 sec)

MySQL (cochesdb)>
```



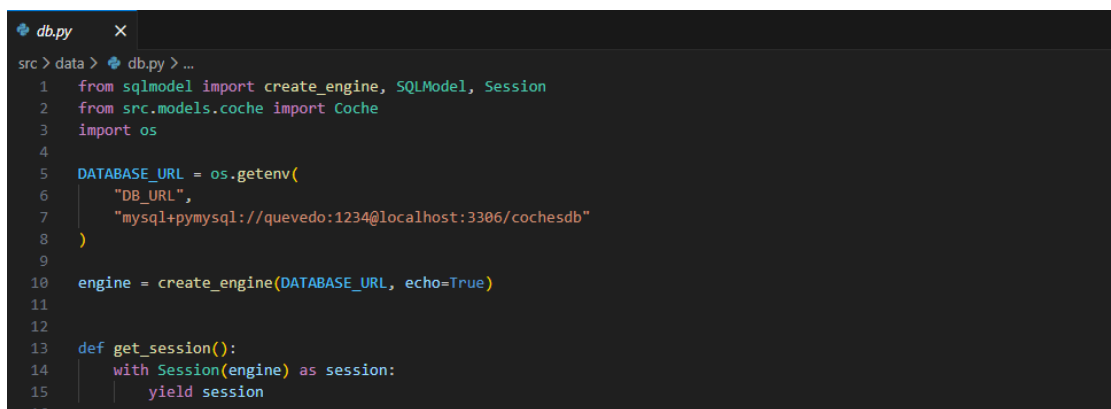
```
MySQL (cochesdb)>SELECT * FROM coche;
+----+-----+-----+-----+
| id | modelo | año  | precio |
+----+-----+-----+-----+
| 1  | Audi A1 | 2021 | 22000  |
| 2  | Audi A3 | 2022 | 28000  |
| 3  | Audi A4 | 2023 | 39000  |
| 4  | Audi A5 | 2023 | 45000  |
| 5  | Audi A6 | 2021 | 52000  |
| 6  | Audi A7 | 2022 | 68000  |
| 7  | Audi A8 | 2023 | 89000  |
| 8  | Audi Q3 | 2022 | 35000  |
+----+-----+-----+-----+
8 rows in set (0.00 sec)
```

En esta parte del proyecto cumplimos con el segundo punto porque logramos conectar nuestra aplicación con una base de datos MySQL real, utilizando tanto una instalación local portátil de MySQL como un contenedor Docker, y en ambos casos la conexión funcionó correctamente. Sustituimos completamente la lista en memoria por un sistema de persistencia profesional, implementando el motor de conexión, configurando las sesiones, creando las tablas mediante SQLAlchemy y adaptando todas las operaciones del CRUD para que trabajen directamente sobre la base de datos. Con esto demostramos que somos capaces de integrar FastAPI con MySQL y gestionar datos reales de forma estable y coherente, independientemente de cómo se despliegue la base de datos.

3. HACER QUE FUNCIONE CON LA BASE DE DATOS MYSQL EN UN CONTENEDOR Y LA APP EN OTRO.

db.py

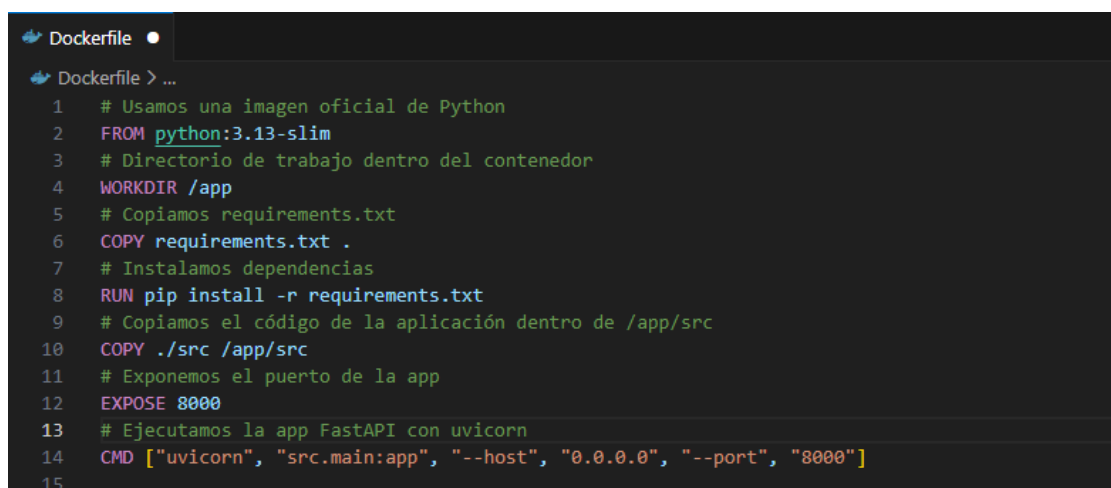
En db.py ajustamos la cadena de conexión para que no dependa siempre de localhost, sino de una variable de entorno llamada DB_URL. Esto era necesario porque cuando la aplicación vive dentro de un contenedor Docker, ya no puede conectarse a MySQL usando localhost, ya que cada contenedor tiene su propio “mundo”. Al usar DB_URL, permitimos que la app se conecte a la base de datos correcta según el entorno: en local usamos localhost, pero en Docker usamos el nombre del servicio (fastapi-db). Este cambio hace que la aplicación sea flexible, portable y compatible con cualquier despliegue.



```
db.py  X
src > data > db.py > ...
1  from sqlmodel import create_engine, SQLModel, Session
2  from src.models.coche import Coche
3  import os
4
5  DATABASE_URL = os.getenv(
6      "DB_URL",
7      "mysql+pymysql://quevedo:1234@localhost:3306/cochesdb"
8  )
9
10 engine = create_engine(DATABASE_URL, echo=True)
11
12
13 def get_session():
14     with Session(engine) as session:
15         yield session
16
```

Dockerfile

El Dockerfile define paso a paso cómo construir la imagen de nuestra aplicación FastAPI. En él instalamos Python, copiamos las dependencias, instalamos los paquetes necesarios y copiamos el código fuente dentro del contenedor. Finalmente, indicamos que la aplicación debe ejecutarse con uvicorn escuchando en el puerto 8000. En resumen, el Dockerfile convierte nuestro proyecto en una imagen lista para ejecutarse en cualquier máquina sin necesidad de instalar nada más.

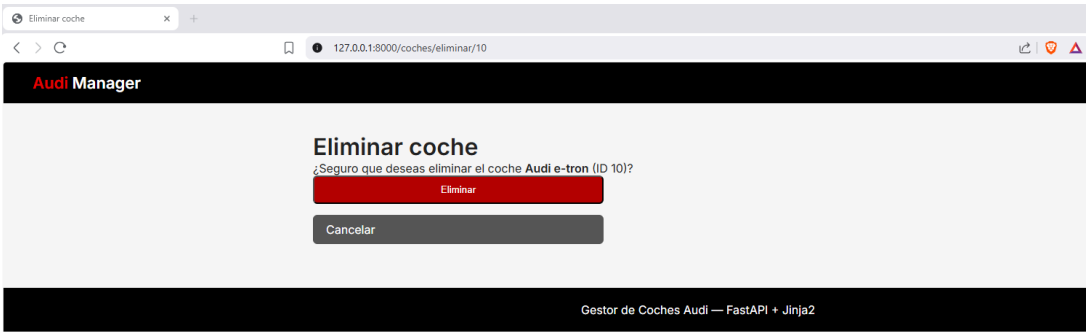
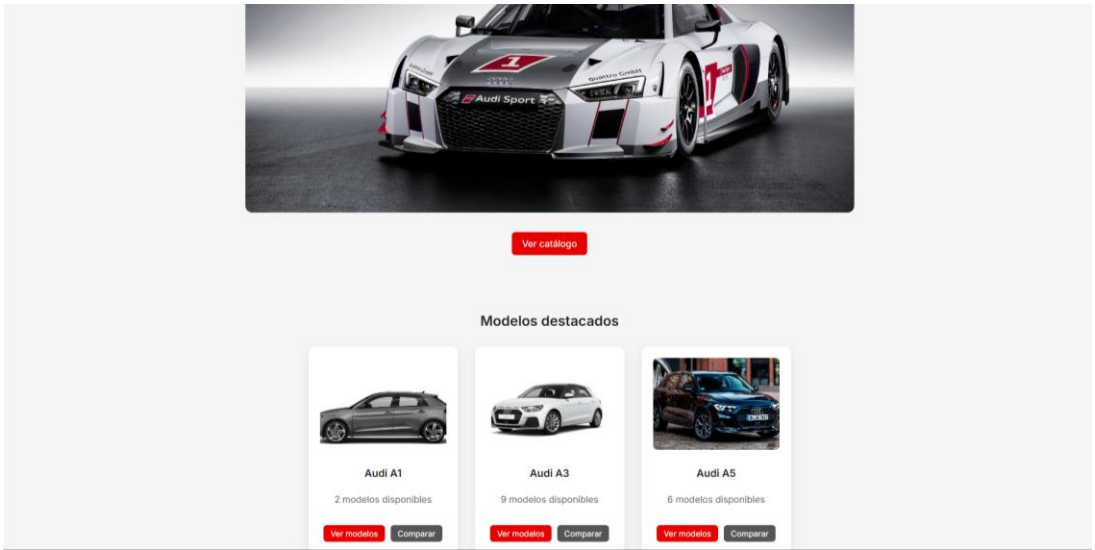


```
Dockerfile  ●
Dockerfile > ...
1  # Usamos una imagen oficial de Python
2  FROM python:3.13-slim
3  # Directorio de trabajo dentro del contenedor
4  WORKDIR /app
5  # Copiamos requirements.txt
6  COPY requirements.txt .
7  # Instalamos dependencias
8  RUN pip install -r requirements.txt
9  # Copiamos el código de la aplicación dentro de /app/src
10 COPY ./src /app/src
11 # Exponemos el puerto de la app
12 EXPOSE 8000
13 # Ejecutamos la app FastAPI con uvicorn
14 CMD ["uvicorn", "src.main:app", "--host", "0.0.0.0", "--port", "8000"]
15
```

docker-compose.yml

El archivo docker-compose.yml es el encargado de orquestar varios contenedores para que trabajen juntos. En nuestro caso, define dos servicios: uno para la aplicación FastAPI y otro para la base de datos MySQL. También establece las variables de entorno, los puertos expuestos, el orden de arranque y la red interna que permite que ambos contenedores se comuniquen. Gracias a este archivo, podemos levantar toda la infraestructura completa con un solo comando, de forma consistente y reproducible.

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS BRUND LOGS
[+] Building 2.9s (12/12) FINISHED
(.venv) PS C:\Users\Jearpearre\Documents\ASIR\IT\DPAPLICACIONES\B\Proyectos\COCHES-API-MEB-MYSQL> docker compose up -d --build
[+] Building 3.5s (12/12) FINISHED
=> [internal] load local bake definitions
=> reading from stdin 678B
=> [internal] load build definition from Dockerfile
=> transferring dockerfile: 51.8B
=> [internal] load metadata for docker.io/library/python:3.11-slim
=> [internal] load .dockerignore
=> transferring context: 2B
[1/5] FROM docker.io/library/python:3.11-slim@sha256:51e1a9317f866178dc791bbee63fac527c82f43958ef74a34e178cf68b18
=> resolve docker.io/library/python:3.11-slim@sha256:51e1a9317f866178dc791bbee63fac527c82f43958ef74a34e178cf68b18
=> [internal] load build context
=> transferring context: 1.51kB
=> CACHED [2/5] WORKDIR /app
=> CACHED [3/5] COPY requirements.txt
=> CACHED [4/5] RUN pip install -r requirements.txt
=> CACHED [5/5] COPY ./src /app/src
=> exporting layers
=> exporting manifest sha256:af5f5af1021e1de5a74a384e5cef73ca35764ba3d7866570e828b7ef7b01ba21
=> exporting config sha256:4863342dc49fe5470f9e36f8915344ba96b2aade107a1a81eb2c8b37d519aa0
=> exporting attestation manifest sha256:8bc5f731f42388ef6a5b0e7f7d666a9c10b01275d1422c2cd3f7f1b8744
=> exporting manifest list sha256:3ab0b3ded7880ac7f3e5b0c7a66a85979e11054cc20bc31a21e4d86d8c3324
=> naming to docker.io/library/fastapi-coches-fastapi-app:latest
=> unpacking to docker.io/library/fastapi-coches-fastapi-app:latest
=> resolving provenance for metadata file
[+] Building 3/0
✓ fastapi-coches-fastapi-app Built
✓ Container fastapi-db Started
(.venv) PS C:\Users\Jearpearre\Documents\ASIR\IT\DPAPLICACIONES\B\Proyectos\COCHES-API-MEB-MYSQL> docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                               NAMES
c8394f232464   fastapi-coches-fastapi-app          "uvicorn src.main:ap..." About a minute Up About a minute    0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp   fastapi-app-coches
95dc87bdc8bf   mysql:8                               "docker-entrypoint.s..." About a minute Up About a minute    0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp   fastapi-db
(.venv) PS C:\Users\Jearpearre\Documents\ASIR\IT\DPAPLICACIONES\B\Proyectos\COCHES-API-MEB-MYSQL> []
```



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS BRUNO LOGS
PS C:\Users\Jeanpierre\Documents\ASIR\II\IMPPLICACIONES\Proyectos\COCHES-API-MEB-MYSQL> & C:\Users\Jeanpierre\Documents\ASIR\II\IMPPLICACIONES\Proyectos\COCHES-API-MEB-MYSQL\venv\Scripts\Activate.ps1
(.venv) PS C:\Users\Jeanpierre\Documents\ASIR\II\IMPPLICACIONES\Proyectos\COCHES-API-MEB-MYSQL> docker exec -it fastapi-db mysql -u quevedo -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 9
Server version: 8.4.8 MySQL Community Server - GPL

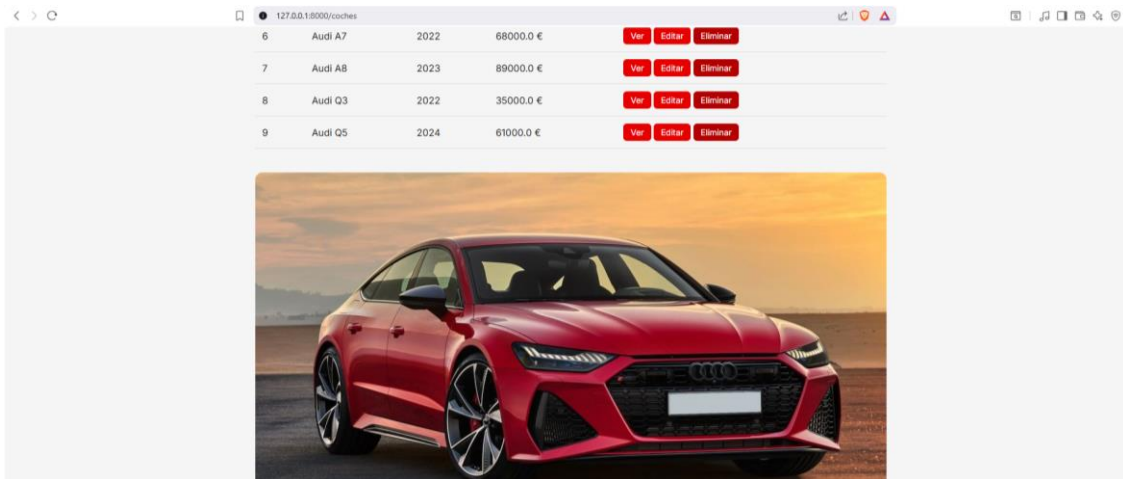
Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> USE cochesdb;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> SELECT * FROM coche;
+----+-----+-----+-----+
| Id | modelo | año | precio |
+----+-----+-----+-----+
| 1 | Audi A1 | 2021 | 22000 |
| 2 | Audi A3 | 2022 | 28000 |
| 3 | Audi A4 | 2023 | 39000 |
| 4 | Audi A5 | 2023 | 45000 |
| 5 | Audi A6 | 2021 | 52000 |
| 6 | Audi A7 | 2022 | 68000 |
| 7 | Audi A8 | 2023 | 89000 |
| 8 | Audi Q3 | 2022 | 35000 |
| 9 | Audi Q5 | 2024 | 61000 |
+----+-----+-----+-----+
9 rows in set (0.00 sec)
```



En esta parte del proyecto cumplimos con el tercer punto porque hemos desplegado la aplicación FastAPI y la base de datos MySQL en contenedores separados, logrando que la app se conecte a la base de datos mediante la red interna de Docker usando el host fastapi-db. Con esta configuración demostramos que podemos aislar servicios, mantener la persistencia de datos y orquestar múltiples contenedores de manera profesional, asegurando que la aplicación funcione de forma estable y reproducible en cualquier entorno sin depender de instalaciones locales.

Podemos imaginarlo como un restaurante modular: la cocina (MySQL) está en un contenedor separado, con todos sus ingredientes y utensilios, y el camarero y la caja (FastAPI) están en otro contenedor, atendiendo a los clientes y recibiendo pedidos. Ambos módulos no necesitan estar en la misma habitación, pero se comunican por un “tubo interno de pedidos” (la red de Docker). Cuando un cliente pide un plato, el camarero envía la orden por el tubo, la cocina la procesa y devuelve la respuesta. Nuestra prueba con éxito fue como hacer un pedido real y ver que la cocina lo preparaba correctamente: la app solicitó datos, la base de datos los entregó y todo funcionó de manera estable y coordinada.

4. HACER QUE FUNCIONE CON UNA BASE DE DATOS POSTGRESQL EN LUGAR DE MYSQL.

requirements.txt

En requirements.txt añadimos el paquete psycopg2-binary, que es el controlador necesario para que SQLAlchemy puedan comunicarse con una base de datos PostgreSQL. Este cambio era imprescindible porque la aplicación estaba preparada inicialmente para MySQL y utilizaba el driver pymysql. Al migrar a PostgreSQL, necesitamos un driver compatible con este motor, y psycopg2-binary es el estándar más utilizado y estable. Gracias a este ajuste, la aplicación puede conectarse correctamente al nuevo motor de base de datos tanto en local como dentro de contenedores Docker.

db.py

En db.py modificamos la cadena de conexión por defecto para que utilice el motor postgresql+psycopg2 en lugar de mysql+pymysql. Este cambio era necesario porque la aplicación debe saber a qué tipo de base de datos conectarse y qué driver utilizar. Además, mantenemos la variable de entorno DB_URL para que Docker Compose pueda inyectar la URL correcta cuando la aplicación se ejecuta dentro de un contenedor. Con este ajuste, la lógica de conexión se vuelve flexible y permite que la app funcione sin cambios adicionales tanto en local como en entornos dockerizados.

```
db.py x
src > data > db.py > ...
1  from sqlalchemy import create_engine, SQLAlchemy, Session
2  from src.models.coche import Coche
3  import os
4
5  DATABASE_URL = os.getenv(
6      "DB_URL",
7      "postgresql+psycopg2://quevedo:1234@localhost:5432/cochesdb"
8  )
9
10 engine = create_engine(DATABASE_URL, echo=True)
11
12
13 def get_session():
14     with Session(engine) as session:
15         yield session
16
17
18 def init_db():
19     # SQLAlchemy.metadata.drop_all(engine)
20     SQLAlchemy.metadata.create_all(engine)
21
22     with Session(engine) as session:
23         coches_existentes = session.query(Coche).count()
24         if coches_existentes == 0:
25             session.add_all([
26                 Coche(modelo="Audi A1", anio=2021, precio=22000),
27                 Coche(modelo="Audi A3", anio=2022, precio=28000),
28                 Coche(modelo="Audi A4", anio=2023, precio=39000),
29                 Coche(modelo="Audi A5", anio=2023, precio=45000),
30                 Coche(modelo="Audi A6", anio=2021, precio=52000),
```

Dockerfile

En el Dockerfile no realizamos cambios profundos, pero sí nos aseguramos de que la instalación de dependencias incluya el nuevo driver de PostgreSQL (psycopg2-binary) y de que la estructura del contenedor copie correctamente la carpeta src para que los imports funcionen. Este archivo define cómo se construye la imagen de la aplicación FastAPI, instalando todas las dependencias necesarias y ejecutando el servidor con uvicorn. Gracias a este Dockerfile, la aplicación puede ejecutarse de forma consistente en cualquier entorno, incluyendo la nueva configuración con PostgreSQL.

```
Dockerfile X
Dockerfile > ...
1 # Usamos una imagen oficial de Python
2 FROM python:3.13-slim
3 # Directorio de trabajo dentro del contenedor
4 WORKDIR /app
5 # Copiamos requirements.txt
6 COPY requirements.txt .
7 # Instalamos dependencias
8 RUN pip install -r requirements.txt
9 # Copiamos el código de la aplicación dentro de /app/src
```

docker-compose.yml

En docker-compose.yml sustituimos el servicio de base de datos MySQL por un servicio basado en la imagen oficial postgres:16, y actualizamos la variable de entorno DB_URL del servicio de la aplicación para que apunte a PostgreSQL usando el host fastapi-db y el puerto 5432. También configuramos las variables POSTGRES_DB, POSTGRES_USER y POSTGRES_PASSWORD para crear automáticamente la base de datos y el usuario dentro del contenedor. Este cambio era fundamental para que la aplicación pudiera comunicarse con PostgreSQL dentro de la red interna de Docker, garantizando que ambos contenedores trabajen juntos de forma correcta y estable.

```
docker-compose.yml X
docker-compose.yml > {} services > {} fastapi-db > {} ports
docker-compose.yml - The Compose specification establishes a standard for the definition of multi-container platform-agnostic applications (compose-spec/json)
1 name: fastapi-coches
2
3 Run All Services
4 services:
5   Run Service
6   fastapi-app:
7     build:
8       context: .
9       dockerfile: Dockerfile
10    container_name: fastapi-app-coches
11    restart: always
12    ports:
13      - "8000:8000"
14    depends_on:
15      - fastapi-db
16    environment:
17      DB_URL: "postgresql+psycopg2://quevedo:1234@fastapi-db:5432/cochesdb"
18
19 Run Service
20 fastapi-db:
21   image: postgres:16
22   container_name: fastapi-db
23   environment:
24     POSTGRES_DB: cochesdb
25     POSTGRES_USER: quevedo
```

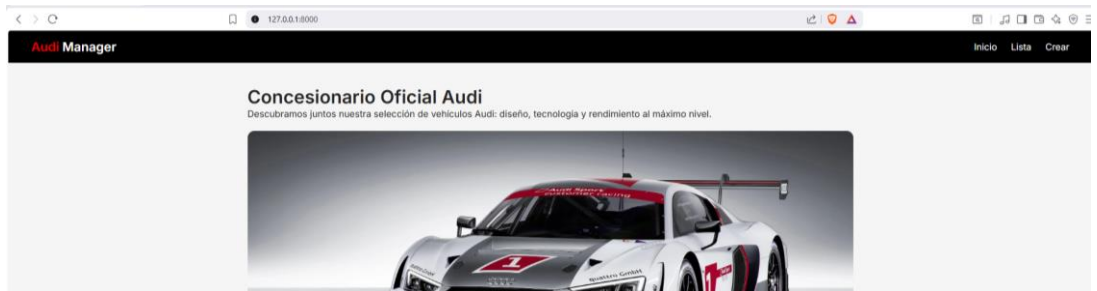
```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS BRUNO LOGS
(.venv) PS C:\Users\Jeanpearre\Documents\ASIR\II\IMPAPLICACIONESWEB\Proyectos\COCHES-API-WEB-MYSQL> docker compose
up -d --build
-> [internal] load .dockerignore 0.0s
-> transferring context: 2B 0.0s
-> [1/5] FROM docker.io/library/python:3.13-slim@sha256:51e1a0a317fdb6e178dc791bbeae63fac5272c82f43958ef74a 0.3s
-> resolve docker.io/library/python:3.13-slim@sha256:51e1a0a317fdb6e178dc791bbeae63fac5272c82f43958ef74a 0.3s
-> [internal] load build context 0.1s
-> transferring context: 2.9kB 0.1s
-> CACHED [2/5] MONGODB /app 0.0s
-> [3/5] COPY requirements.txt . 1.1s
-> [4/5] RUN pip install -r requirements.txt 49.4s
-> [5/5] COPY ./src /app/src 0.3s
-> exporting to image 23.7s
-> exporting layers 18.8s
-> exporting manifest sha256:91002e7c4d428a30a47f68dc4af92d4392191a8352b9a0bba46c224c858f775 0.0s
-> exporting config sha256:53d04815b5da0157d27ae614479734a32ea71a543757ef3c841a03934728e53 0.1s
-> exporting attestation manifest sha256:bac6c5658ac524423ea37c35f908017924c54cf0de9209fa5d811384c99ff 0.1s
-> exporting manifest list sha256:3c9521faf5835fcf604872435608ba2564d2a2c015ba265411cee7cc61effb 0.0s
-> naming to docker.io/library/fastapi-coches-fastapi-app:latest 0.0s
-> unpacking to docker.io/library/fastapi-coches-fastapi-app:latest 4.4s
-> resolving provenance for metadata file 0.2s
[+] Running 3/3
✓ fastapi-coches-fastapi-app Built 0.0s
✓ Container fastapi-db Started 10.7s
✓ Container fastapi-app-coches Started 11.7s
(.venv) PS C:\Users\Jeanpearre\Documents\ASIR\II\IMPAPLICACIONESWEB\Proyectos\COCHES-API-WEB-MYSQL> docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
d77e0f2a2259 fastapi-coches-fastapi-app "uvicorn src.main:ap..." 34 seconds ago Up 15 seconds 0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp fastapi-app-coches
4f37b6595a32 postgres:16 "docker-entrypoint.s..." 35 seconds ago Up 25 seconds 0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp fastapi-db
(.venv) PS C:\Users\Jeanpearre\Documents\ASIR\II\IMPAPLICACIONESWEB\Proyectos\COCHES-API-WEB-MYSQL>
```

```
(.venv) PS C:\Users\Jeanpearre\Documents\ASIR\II\IMPAPLICACIONESWEB\Proyectos\COCHES-API-WEB-MYSQL> docker exec -it fastapi-db bash
root@4f37b6595a32:/# psql -U quevedo -d cochesdb
psql (16.11 (Debian 16.11-1.pgdg13+1))
Type "help" for help.

cochesdb=# \dt
          List of relations
 Schema | Name | Type | Owner
-----+-----+-----+-----
 public | coche | table | quevedo
(1 row)

cochesdb=# SELECT * FROM coche;
 id | modelo | anio | precio
-----+-----+-----+-----
  1 | Audi A1 | 2021 | 22000
  2 | Audi A3 | 2022 | 28000
  3 | Audi A4 | 2023 | 39000
  4 | Audi A5 | 2023 | 45000
  5 | Audi A6 | 2021 | 52000
  6 | Audi A7 | 2022 | 68000
  7 | Audi A8 | 2023 | 89000
  8 | Audi Q3 | 2022 | 35000
  9 | Audi Q5 | 2024 | 61000
 10 | Audi e-tron | 2023 | 72000
(10 rows)

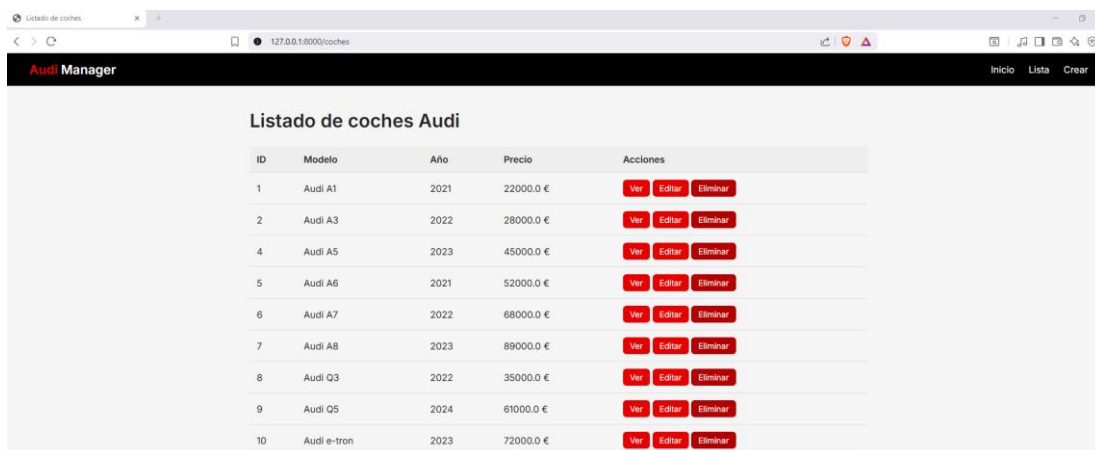
cochesdb=#
```



Listado de coches Audi				
ID	Modelo	Año	Precio	Acciones
1	Audi A1	2021	22000.0 €	Ver Editar Eliminar
2	Audi A3	2022	28000.0 €	Ver Editar Eliminar
3	Audi A4	2023	39000.0 €	Ver Editar Eliminar
4	Audi A5	2023	45000.0 €	Ver Editar Eliminar
5	Audi A6	2021	52000.0 €	Ver Editar Eliminar
6	Audi A7	2022	68000.0 €	Ver Editar Eliminar
7	Audi A8	2023	89000.0 €	Ver Editar Eliminar
8	Audi Q3	2022	35000.0 €	Ver Editar Eliminar
9	Audi Q5	2024	61000.0 €	Ver Editar Eliminar
10	Audi e-tron	2023	72000.0 €	Ver Editar Eliminar

```
cochesdb=# SELECT * FROM coche;
 id | modelo | anio | precio
-----+-----+-----+-----
  1 | Audi A1 | 2021 | 22000
  2 | Audi A3 | 2022 | 28000
  3 | Audi A4 | 2023 | 39000
  4 | Audi A5 | 2023 | 45000
  5 | Audi A6 | 2021 | 52000
  6 | Audi A7 | 2022 | 68000
  7 | Audi A8 | 2023 | 89000
  8 | Audi Q3 | 2022 | 35000
  9 | Audi Q5 | 2024 | 61000
 10 | Audi e-tron | 2023 | 72000
(10 rows)

cochesdb=# DELETE FROM coche WHERE id = 3;
DELETE 1
cochesdb=# SELECT * FROM coche;
 id | modelo | anio | precio
-----+-----+-----+-----
  1 | Audi A1 | 2021 | 22000
  2 | Audi A3 | 2022 | 28000
  4 | Audi A5 | 2023 | 45000
  5 | Audi A6 | 2021 | 52000
  6 | Audi A7 | 2022 | 68000
  7 | Audi A8 | 2023 | 89000
  8 | Audi Q3 | 2022 | 35000
  9 | Audi Q5 | 2024 | 61000
 10 | Audi e-tron | 2023 | 72000
(9 rows)
```



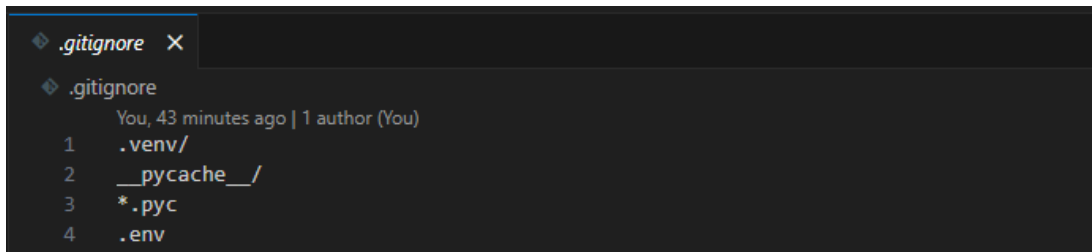
En esta parte del proyecto cumplimos con el cuarto punto porque hemos sustituido completamente MySQL por PostgreSQL, adaptando la URL de conexión, las dependencias y la configuración de los contenedores para que la aplicación funcione sobre este nuevo motor de base de datos sin modificar la lógica del proyecto. Además, hemos eliminado un artículo directamente desde la base de datos y comprobado que el cambio se refleja correctamente en la interfaz web, lo que confirma que la comunicación entre la aplicación, el contenedor de PostgreSQL y las operaciones CRUD funciona de manera estable y coherente en este nuevo entorno.

5. HACER QUE FUNCIONE CON POSTGRESQL EN UNA NUBE RENDER. DOCUMENTAR EL PROCESO DE DESPLIEGUE EN UN FICHERO MARKDOWN.

En este quinto punto vamos a desplegar nuestra aplicación FastAPI en la nube utilizando Render, conectándola a una base de datos PostgreSQL también alojada en Render. Para lograrlo, primero subiremos nuestro proyecto a GitHub y lo prepararemos para que Render pueda construirlo automáticamente. Luego crearemos un servicio de base de datos PostgreSQL en Render y configuraremos la variable de entorno DB_URL para que nuestra aplicación se conecte correctamente. A continuación, crearemos un nuevo servicio web en Render que se vincule directamente con nuestro repositorio de GitHub, definiendo los comandos de instalación y arranque. Finalmente, verificaremos que la aplicación funciona correctamente en la web y que cada vez que hagamos un git push, Render detecte los cambios y redepliegue la aplicación automáticamente. Todo el proceso será documentado paso a paso para garantizar que sea reproducible y verificable.

.gitignore

El archivo .gitignore es un componente fundamental de cualquier proyecto versionado con Git, porque nos permite definir qué archivos y carpetas no deben incluirse en el repositorio. En este punto lo utilizamos para mantener nuestro proyecto limpio y profesional, evitando subir elementos temporales, archivos generados automáticamente, credenciales locales o cualquier contenido que no forme parte del código real de la aplicación.



```
.gitignore
You, 43 minutes ago | 1 author (You)
1 .venv/
2 __pycache__/
3 *.pyc
4 .env
```

git init

git add .

git commit -m "Proyecto FastAPI_PostgreSQL"

Este conjunto de comandos representa el punto de partida para versionar nuestro proyecto con Git, y lo utilizamos porque necesitamos tener el código organizado, controlado y listo para enviarlo a GitHub antes de desplegarlo en Render. Primero inicializamos el repositorio con git init. A continuación añadimos todos los archivos del proyecto con git add ., incorporándolos al área de preparación. Finalmente, realizamos nuestro primer commit con un mensaje descriptivo, creando una instantánea estable del estado inicial del proyecto. Hacemos todo esto porque Render se integra directamente con GitHub, y tener un repositorio bien configurado es esencial para que el despliegue automático funcione sin problemas.

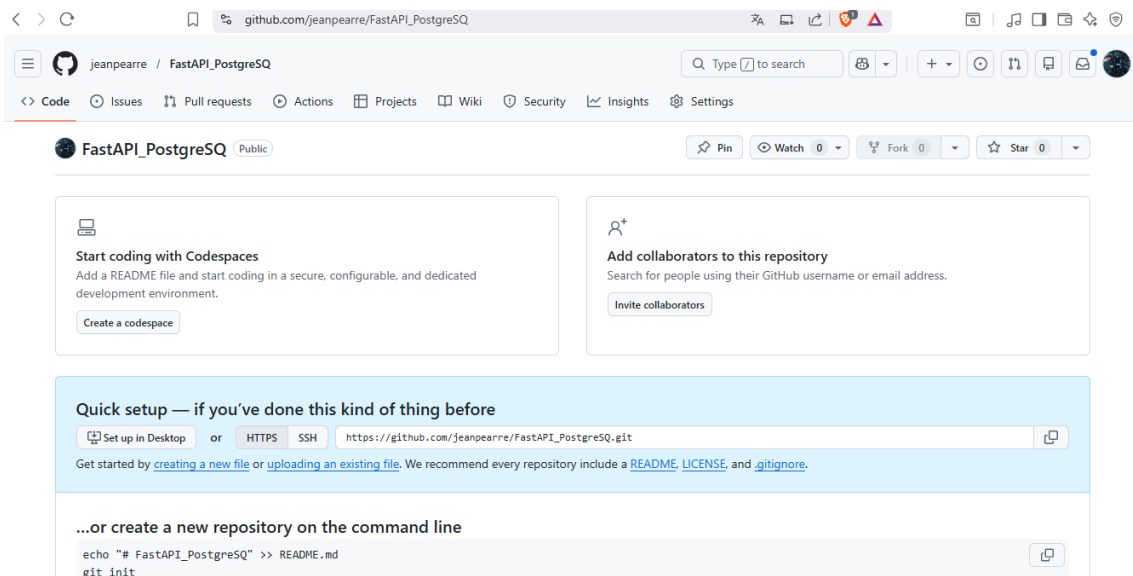
```

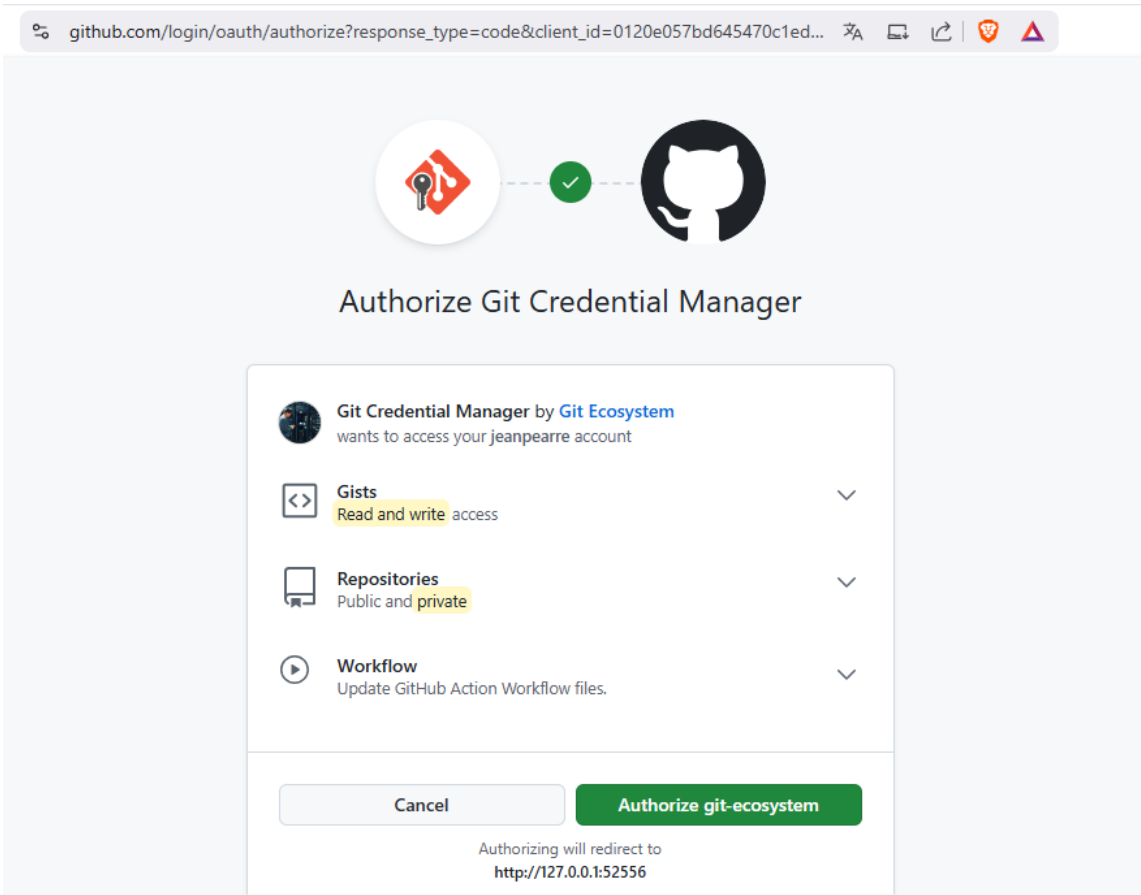
create mode 100644 requirements.txt
(.venv) PS C:\Users\Jeanpearre\Documents\ASIR\II\IMPAPLICACIONESWEB\Proyectos\COCHES-API-WEB-MYSQL> git config --global user.email "j.leonsipion@email.com"
(.venv) PS C:\Users\Jeanpearre\Documents\ASIR\II\IMPAPLICACIONESWEB\Proyectos\COCHES-API-WEB-MYSQL> git commit -m "Proyecto FastAPI_PostgreSQL"
[master (root-commit) 1cb2745] Proyecto FastAPI_PostgreSQL
27 files changed, 1772 insertions(+)
create mode 100644 .gitignore
create mode 100644 COCHES-API-WEB-MYSQL.docx
create mode 100644 COCHES-API-WEB-MYSQL.txt
create mode 100644 Dockerfile
create mode 100644 .gitignore
create mode 100644 COCHES-API-WEB-MYSQL.docx
create mode 100644 COCHES-API-WEB-MYSQL.txt
create mode 100644 Dockerfile
create mode 100644 docker-compose-mysql.yml
create mode 100644 docker-compose.yml
create mode 100644 Dockerfile
create mode 100644 docker-compose-mysql.yml
create mode 100644 docker-compose.yml

```

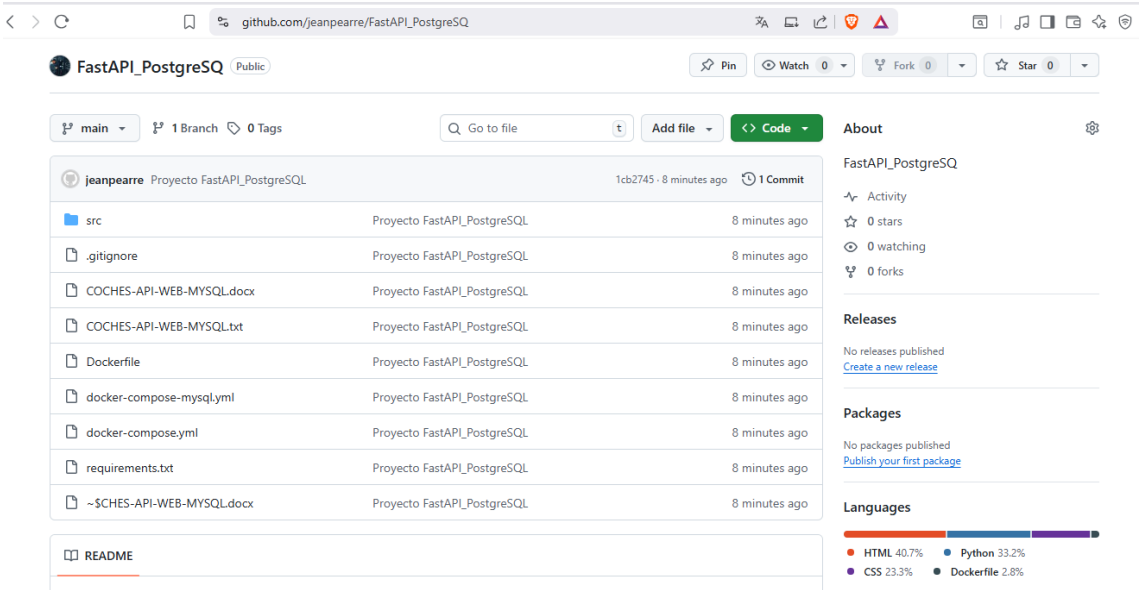
Creamos el repositorio remoto en GitHub y lo vinculamos con nuestro proyecto local para establecer un flujo de trabajo ordenado, seguro y compatible con Render. Al generar un repositorio vacío —sin README, sin .gitignore y sin licencia— garantizamos que el contenido inicial provenga exclusivamente de nuestro proyecto local, evitando conflictos innecesarios y manteniendo un control total sobre la estructura del código. GitHub se convierte así en el punto central donde almacenamos la versión oficial de nuestra aplicación, lo que nos permite trabajar con historial de cambios, revertir errores y colaborar de forma más organizada. Además, tener el proyecto en GitHub es un requisito fundamental para que Render pueda acceder al código y desplegarlo automáticamente en la nube.

Una vez creado el repositorio remoto, lo enlazamos con nuestro proyecto local mediante los comandos `git remote add origin`, `git branch -M main` y `git push -u origin main`. Con esta configuración establecemos la conexión entre ambos entornos y definimos la rama principal desde la que trabajaremos. Al realizar el primer push, enviamos todo el contenido del proyecto a GitHub, creando la base sobre la que Render realizará sus despliegues. Hacemos todo esto porque necesitamos un flujo de despliegue automatizado: cada vez que actualicemos el código y hagamos un `git push`, Render detectará los cambios y reconstruirá la aplicación sin que tengamos que intervenir manualmente. De esta forma, garantizamos un desarrollo más fluido, profesional y fácil de mantener.



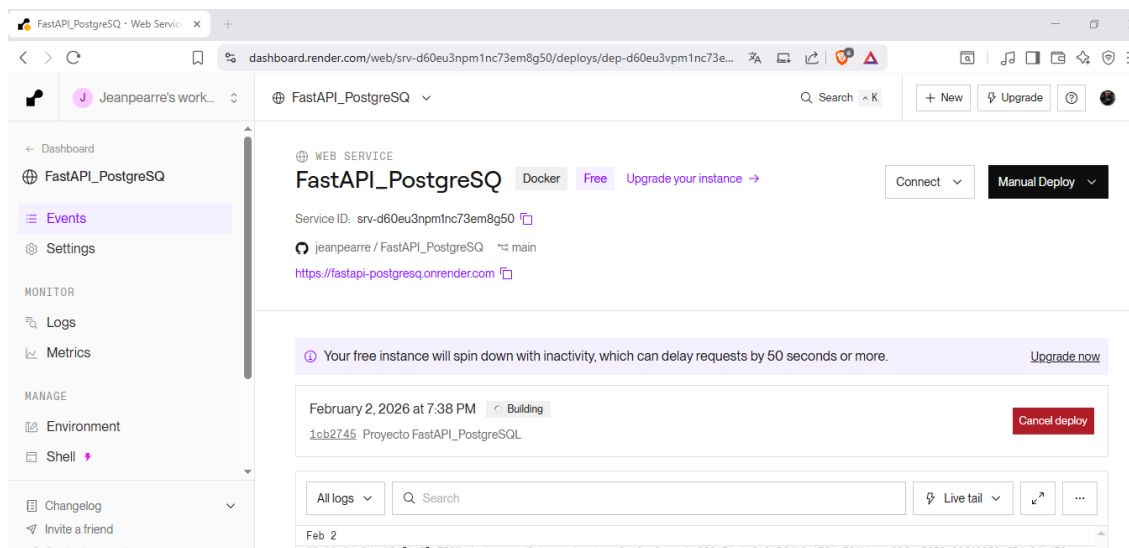
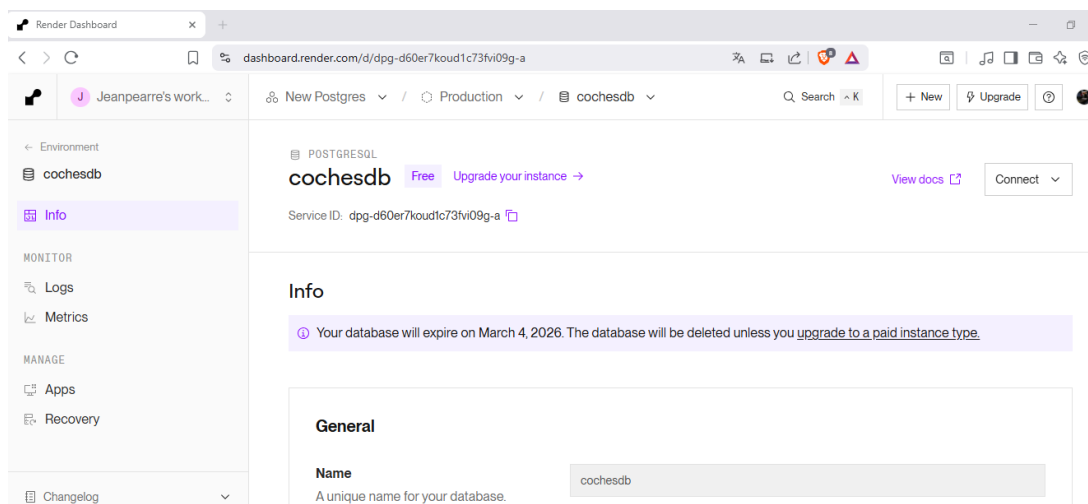


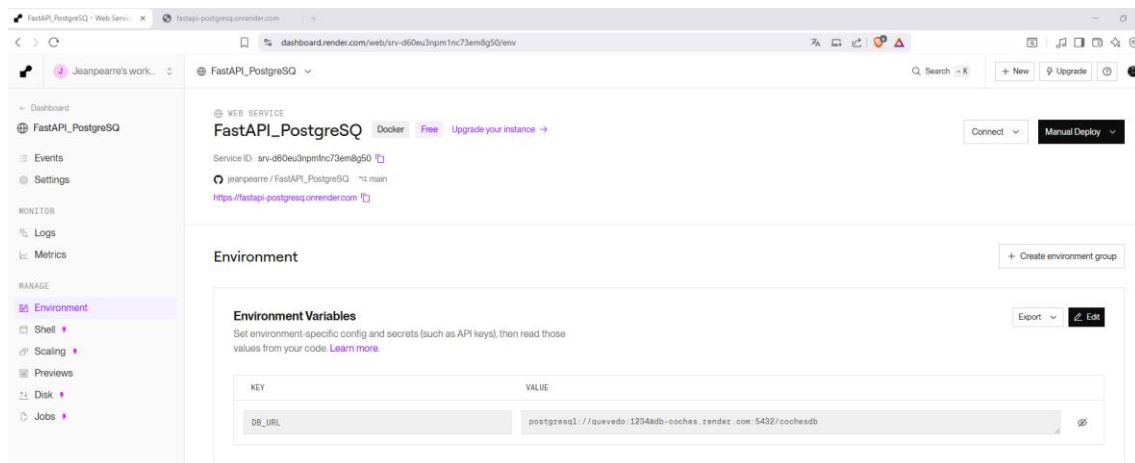
```
nothing to commit, working tree clean
(.venv) PS C:\Users\Jeanpearre\Documents\ASIR\II\IMPAPLICACIONESWEB\Proyectos\COCHES-API-WEB-MYSQL> git remote add origin https://github.com/jeanpearre/FastAPI_PostgreSQL.git
(.venv) PS C:\Users\Jeanpearre\Documents\ASIR\II\IMPAPLICACIONESWEB\Proyectos\COCHES-API-WEB-MYSQL> git branch -M main
(.venv) PS C:\Users\Jeanpearre\Documents\ASIR\II\IMPAPLICACIONESWEB\Proyectos\COCHES-API-WEB-MYSQL> git push -u origin main
info: please complete authentication in your browser...
Enumerating objects: 38, done.
Counting objects: 100% (38/38), done.
Delta compression using up to 4 threads
Compressing objects: 100% (33/33), done.
Writing objects: 100% (38/38), 7.67 MiB | 412.00 KiB/s, done.
Total 38 (delta 9), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (9/9), done.
To https://github.com/jeanpearre/FastAPI_PostgreSQL.git
 * [new branch]    main -> main
branch 'main' set up to track 'origin/main'.
(.venv) PS C:\Users\Jeanpearre\Documents\ASIR\II\IMPAPLICACIONESWEB\Proyectos\COCHES-API-WEB-MYSQL> |
```



Pasamos a Render. Configuramos la base de datos en la nube que utilizará nuestra aplicación, aprovechando que Render ofrece PostgreSQL totalmente gestionado y accesible desde cualquier despliegue. Creamos la base de datos desde el panel, eligiendo un nombre como cochesdb, la región más cercana y el plan gratuito si está disponible. Una vez creada, copiamos la URL de conexión que contiene usuario, contraseña, host, puerto y nombre de la base de datos, que nuestra aplicación usará para conectarse. Hacemos esto para tener una base estable, accesible y persistente, que Render integra fácilmente con el servicio web sin configuraciones complejas.

Luego, desplegamos nuestra aplicación FastAPI en Render utilizando directamente el repositorio de GitHub, lo que permite automatizar el despliegue cada vez que actualizamos el código. Creamos un Web Service, seleccionamos “Build from a Git repository”, conectamos GitHub y elegimos el repositorio fastapi-coches. Configuramos nombre, región, rama main y runtime Python, para que Render instale dependencias y ejecute la aplicación. Añadimos la variable de entorno DB_URL con la URL de PostgreSQL, permitiendo que FastAPI acceda a la base de datos sin exponer credenciales. Finalmente realizamos un despliegue limpio con “Clear build cache & deploy”.

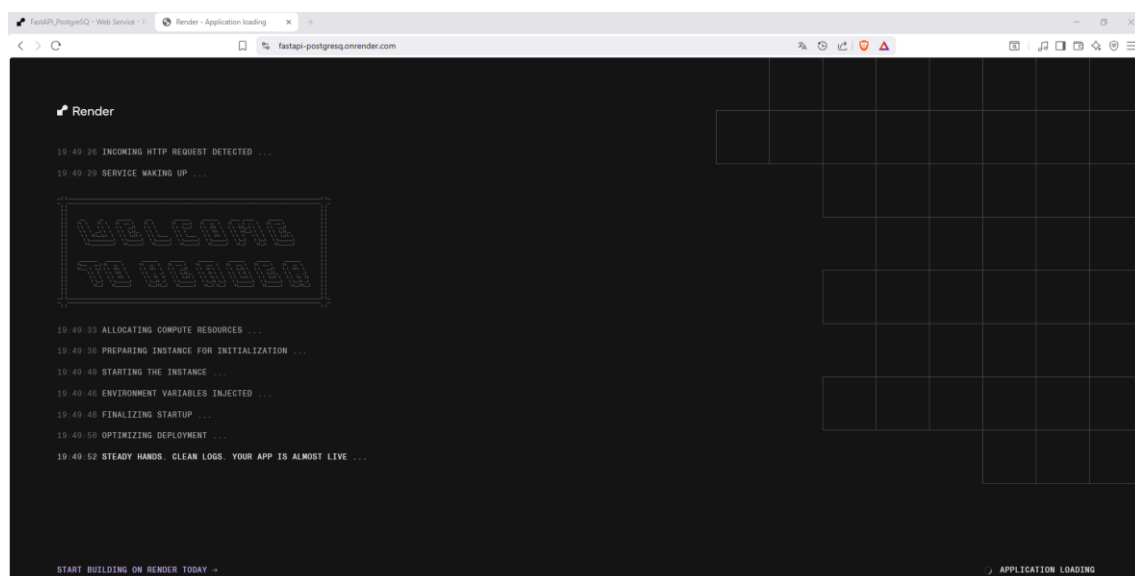




<https://fastapi-postgresql.onrender.com/>
<https://fastapi-postgresql.onrender.com/coches>

Pasamos a validar que nuestra aplicación desplegada en Render funciona correctamente accediendo a las rutas principales que expone el servicio web. Al abrir https://TU_SERVICIO.onrender.com/ comprobamos que el servidor arranca sin errores y que la página inicial se renderiza como esperamos, lo que confirma que el despliegue se ha realizado correctamente y que Render está ejecutando nuestra aplicación FastAPI.

Después visitamos https://TU_SERVICIO.onrender.com/coches, donde verificamos que la aplicación es capaz de conectarse a la base de datos PostgreSQL mediante la variable de entorno DB_URL, recuperar los datos y mostrarlos en pantalla. Realizamos esta validación porque es la forma más directa y fiable de asegurarnos de que tanto el backend como la base de datos están integrados, operativos y funcionando en la nube tal como hemos configurado.



Dashboard

FastAPI_PostgreSQL

Events

Settings

MONITOR

Logs

Metrics

MANAGE

Environment

Shell

Scaling

Previews

Disk

Jobs

FastAPI_PostgreSQL

Free

Upgrade your instance

Connect

Manual Deploy

Service ID: srv-d60eu3npmtnc73em8g50

jeanpearre / FastAPI_PostgreSQL ~1 min

https://fastapi-postgresql.onrender.com

Your free instance will spin down with inactivity, which can delay requests by 50 seconds or more.

Filter events 31

Deploy started for 1cb2745: Proyecto FastAPI_PostgreSQL

Manually triggered by you via Dashboard

Build cache cleared

February 2, 2026 at 8:18 PM

Cancel deploy

Deploy live for 1cb2745: Proyecto FastAPI_PostgreSQL

February 2, 2026 at 8:18 PM

Deploy started for 1cb2745: Proyecto FastAPI_PostgreSQL

Environment updated

February 2, 2026 at 8:16 PM

FastAPI_PostgreSQL - Web Service

jeanpearre/FastAPI_PostgreSQL_FastAPI

Inicio

fastapi-postgresql.onrender.com

Audi Manager

Inicio

Lista

Crear

Concesionario Oficial Audi

Descubramos juntos nuestra selección de vehículos Audi: diseño, tecnología y rendimiento al máximo nivel.



Ver catálogo

Modelos destacados









Ver catálogo

Modelos destacados



Audi A1

2 modelos disponibles

Ver modelos

Comparar



Audi A3

9 modelos disponibles

Ver modelos

Comparar



Audi A5

6 modelos disponibles

Ver modelos

Comparar

Gestor de Coches Audi — FastAPI + Jinja2

git add .

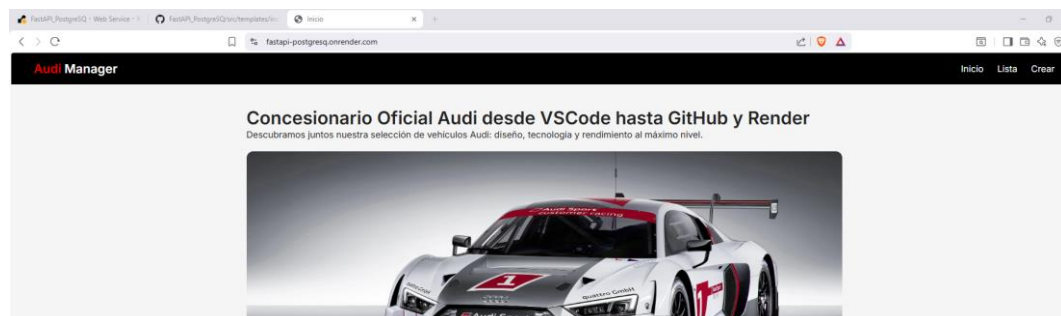
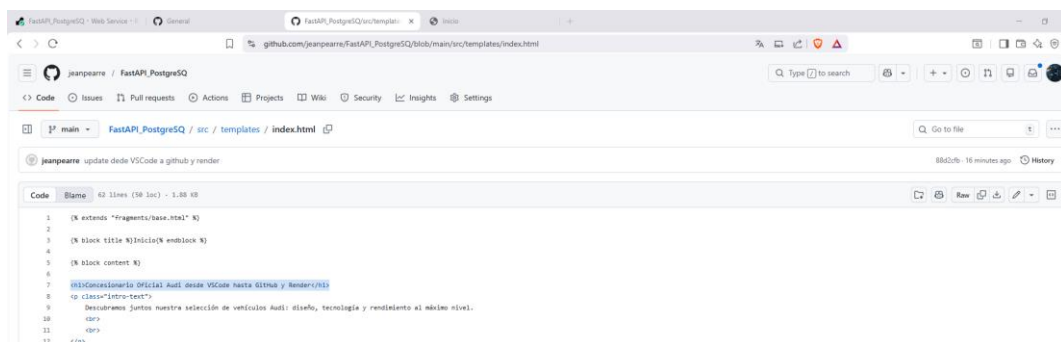
git commit -m "Cambio de texto para probar redeploy en Render"

git push

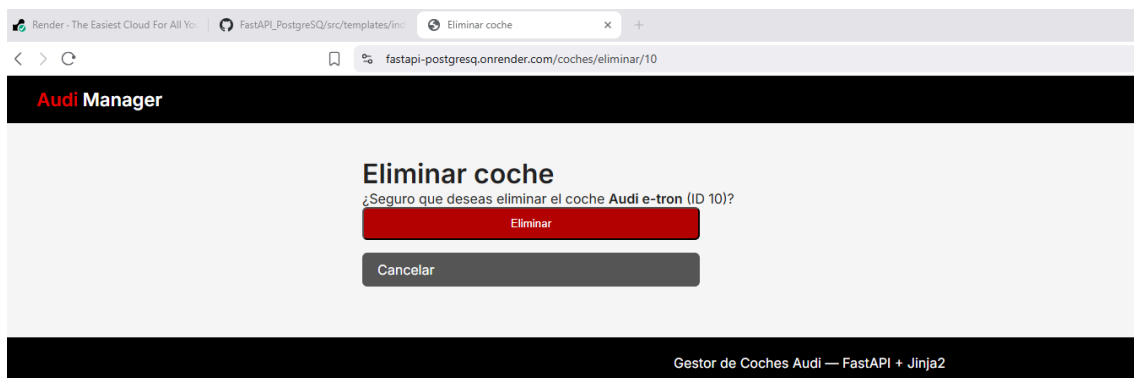
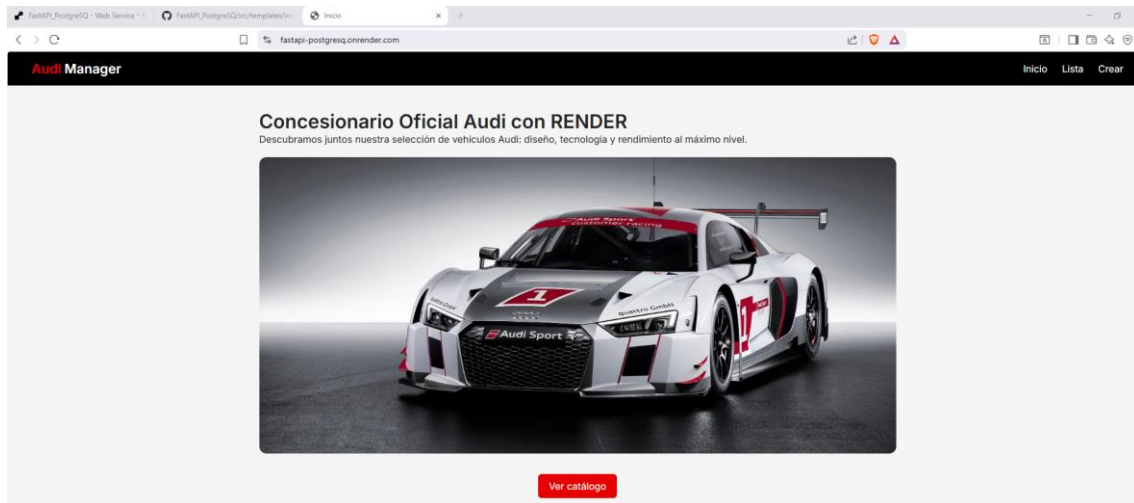
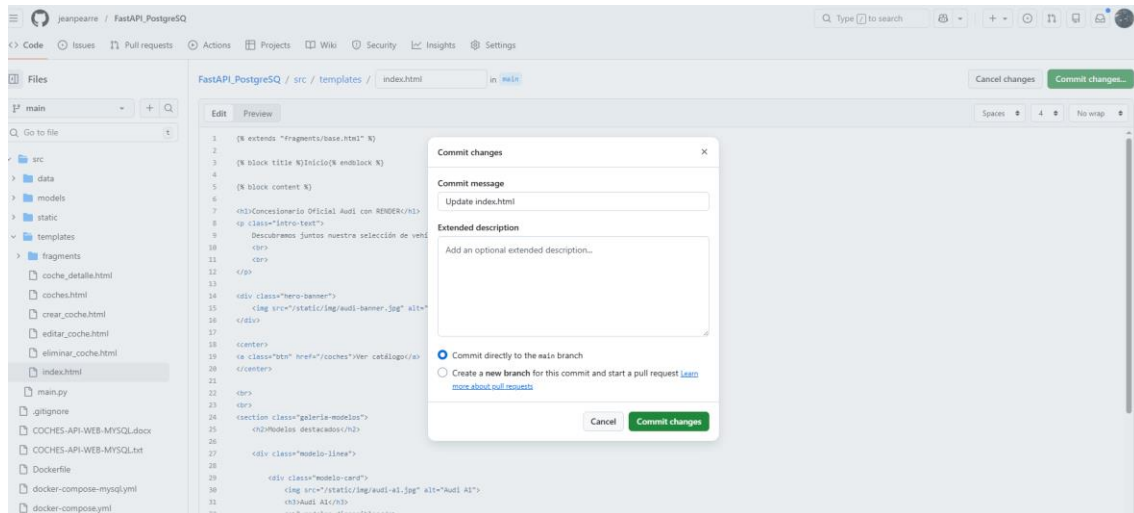
Pasamos a realizar las pruebas de actualización desde nuestro entorno local (VSCode) para comprobar que el despliegue automático en Render funciona correctamente. Ejecutamos los comandos `git add .`, `git commit -m "Cambio de texto para probar redeploy en Render"` y `git push` para enviar una modificación sencilla al repositorio de GitHub. Con este proceso verificamos que Render detecta el nuevo commit, reconstruye la aplicación y publica una versión actualizada sin necesidad de intervención manual. Hacemos esta prueba porque es fundamental asegurarnos de que el flujo de trabajo está bien configurado y que cualquier cambio que realicemos en el código se reflejará automáticamente en la aplicación desplegada en la nube.

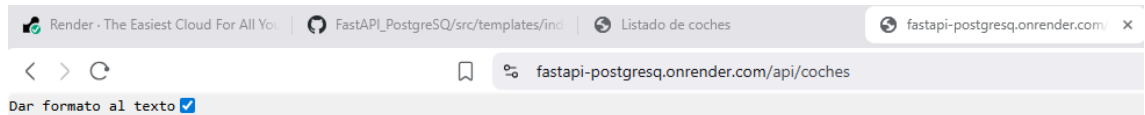
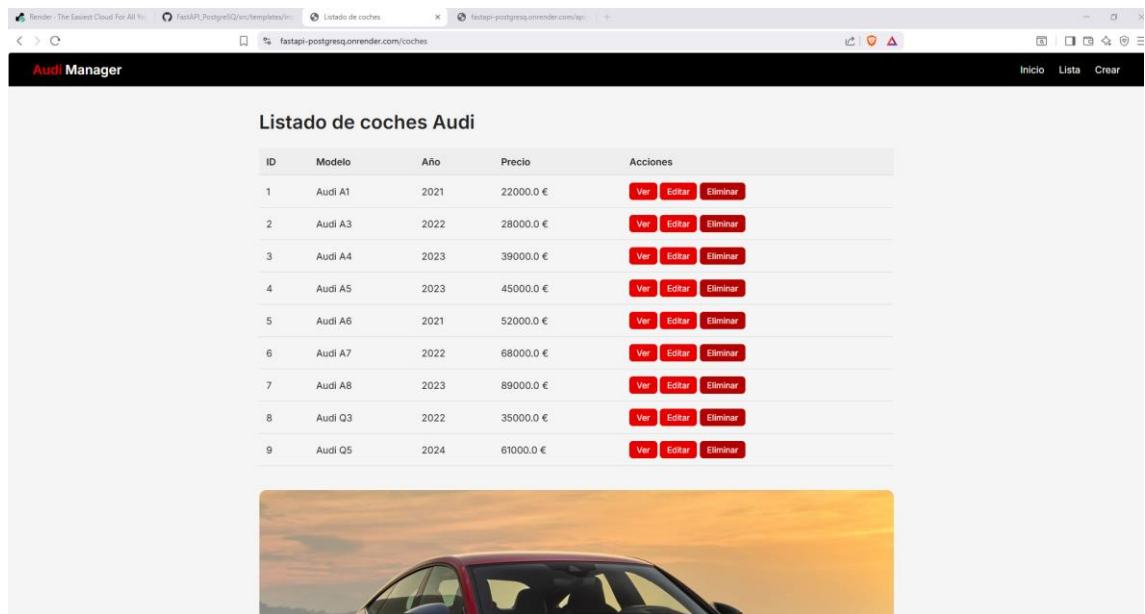
```
index.html X
src > templates > index.html > h1
You, 3 minutes ago | 1 author (You)
1 {% extends "fragments/base.html" %}
2
3 {% block title %}Inicio{% endblock %}
4
5 {% block content %}
6
7 <h1>Concesionario Oficial Audi desde VSCode hasta GitHub y Render</h1> You, 3 minutes ago • Uncommitted changes
8 <p class="intro-text">
9     Descubramos juntos nuestra selección de vehículos Audi: diseño, tecnología y rendimiento al máximo nivel.
10 <br>
11 <br>
12 </p>
13
14 <div class="hero-banner">

(.venv) PS C:\Users\Jeanpearre\Documents\VASIR\II\IMPAPLICACIONESWEB\Proyectos\COCHES-API-WEB-MYSQL> git add .
(.venv) PS C:\Users\Jeanpearre\Documents\VASIR\II\IMPAPLICACIONESWEB\Proyectos\COCHES-API-WEB-MYSQL> git commit -m "update dede VSCode a github y render"
[main 88d2cfb] update dede VSCode a github y render
1 file changed, 1 insertion(+), 1 deletion(-)
(.venv) PS C:\Users\Jeanpearre\Documents\VASIR\II\IMPAPLICACIONESWEB\Proyectos\COCHES-API-WEB-MYSQL> git push
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 4 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (5/5), 461 bytes | 461.00 KiB/s, done.
Total 5 (delta 4), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (4/4), completed with 4 local objects.
To https://github.com/Jeanpearre/FastAPI_PostgreSQL.git
 e8112fa..88d2cfb main -> main
(.venv) PS C:\Users\Jeanpearre\Documents\VASIR\II\IMPAPLICACIONESWEB\Proyectos\COCHES-API-WEB-MYSQL> 
```



Y también editando desde el propio GitHub validamos que el sistema de despliegue automático funciona incluso cuando realizamos cambios directamente en la interfaz web del repositorio. Modificamos un archivo sencillo, como index.html, y confirmamos el cambio con un commit desde GitHub para que Render detecte la actualización sin necesidad de usar VSCode ni la terminal. Hacemos esta prueba porque queremos asegurarnos de que cualquier modificación realizada en el repositorio —ya sea local o remota— desencadena un redeploy automático en Render, garantizando así un flujo de trabajo flexible, fiable y coherente con las necesidades del proyecto.





```
[
  {
    "precio": 22000,
    "id": 1,
    "modelo": "Audi A1",
    "anio": 2021
  },
  {
    "precio": 28000,
    "id": 2,
    "modelo": "Audi A3",
    "anio": 2022
  },
  {
    "precio": 39000,
    "id": 3,
    "modelo": "Audi A4",
    "anio": 2023
  },
  {
    "precio": 45000,
    "id": 4,
    "modelo": "Audi A5",
    "anio": 2023
  },
  {
    "precio": 52000,
    "id": 5,
    "modelo": "Audi A6",
    "anio": 2021
  },
  {
    "precio": 68000,
    "id": 6,
    "modelo": "Audi A7",
    "anio": 2022
  },
  {
    "precio": 89000,
    "id": 7,
    "modelo": "Audi A8",
    "anio": 2023
  },
  {
    "precio": 35000,
    "id": 8,
    "modelo": "Audi Q3",
    "anio": 2022
  },
  {
    "precio": 61000,
    "id": 9,
    "modelo": "Audi Q5",
    "anio": 2024
  }
]
```

A lo largo de este proyecto hemos aprendido a construir una aplicación completa paso a paso, empezando por integrar páginas web con Jinja2 dentro de una API REST para combinar la lógica backend con vistas dinámicas. Después profundizamos en el uso de bases de datos, primero conectando la aplicación a MySQL en localhost y entendiendo cómo gestionar modelos, sesiones y consultas desde SQLAlchemy. Más adelante dimos un salto hacia la contenedorización, separando la base de datos MySQL y la aplicación FastAPI en contenedores independientes, lo que nos permitió comprender cómo se comunican servicios aislados y cómo Docker facilita entornos reproducibles y portables.

También aprendimos a migrar la aplicación de MySQL a PostgreSQL, adaptando la configuración, los modelos y la cadena de conexión para trabajar con un motor de base de datos diferente. Este cambio nos permitió entender las diferencias entre ambos sistemas y reforzar la importancia de diseñar aplicaciones flexibles y desacopladas. Finalmente, completamos el ciclo llevando la aplicación a la nube mediante Render, creando una base de datos PostgreSQL gestionada, configurando variables de entorno, desplegando el servicio web desde GitHub y documentando todo el proceso en un archivo Markdown. Con esto cerramos un recorrido completo que abarca desarrollo local, contenedores, migraciones de base de datos y despliegue en la nube.

Podemos imaginar todo este proyecto como construir un vehículo desde cero. Primero diseñamos el interior y el tablero (las páginas Jinja2), luego instalamos el motor básico en el garaje de casa (MySQL en localhost). Más tarde montamos el motor y la carrocería en módulos separados dentro de un taller profesional (contenedores Docker), y después sustituimos el motor por uno más potente y eficiente (PostgreSQL). Finalmente llevamos el coche a una autopista real, abierta al público, donde puede circular sin interrupciones (Render en la nube). Esta analogía nos ayuda a ver que no solo hemos creado una aplicación, sino que hemos aprendido a diseñarla, montarla, mejorarla y ponerla en producción como haría un equipo profesional de desarrollo.

https://github.com/jeanpearre/FastAPI_PostgreSQL

<https://fastapi-postgresq.onrender.com/>

<https://fastapi-postgresq.onrender.com/coches>

Jeanpearre Leon S.